통합반 6회차 — 낭독 대본

[이론 01] 업무 수행 방법을 재사용 가능한 능력으로 만들기

[SLIDE 01] 업무 지침의 재사용과 외부 시스템 연결

안녕하세요. 이번 회차에서는 에이전트가 일하는 방법, 그리고 그 일에 쓰는 바깥 시스템의 도구를 다룹니다.

쿨러 성능 저하 경보를 맡은 에이전트를 떠올려 볼게요. 이 에이전트는 경보가 올 때마다 같은 방법으로 원인을 찾고, 조치를 비교해야 합니다. 그런데 그 방법은 어디에 적혀 있을까요? 또 그 방법에 적힌 도구는 어떻게 지식 그래프와 업무 DB까지 닿을까요?

이번 회차는 이 두 물음에서 출발합니다. 일하는 방법은 스킬이라는 지침으로 남기고, 그 스킬이 부르는 도구는 MCP라는 표준으로 안전하게 연결하는 데까지 가 보겠습니다.

[SLIDE 02] 학습목표

먼저 이번 회차의 학습목표부터 볼게요. 화면에 카드가 네 장 있습니다.

첫 카드는 에이전트 스킬이에요. 에이전트가 읽는 지침 파일인 스킬을 원문 그대로 읽어 보고, 현장 작업자가 읽는 SOP나 한 번 쓰고 마는 프롬프트와 어떻게 다른지 가려 봅니다. 두 번째 카드는 MCP 연결입니다. 스킬이 부르는 도구가 MCP로 어떻게 이어지는지 설명할 수 있으면 돼요. 도구 목록을 받고, 입력을 맞춰서 부르고, 응답을 읽는 과정이죠.

세 번째 카드는 연결의 안전 기준입니다. 읽기와 쓰기 권한, 파일 범위, 비밀값, 그리고 실패하거나 잘린 응답까지 점검해요. 마지막 카드는 실습입니다. 업무 도구와 여러 MCP 서버를 직접 연결해서 불러 보고, 스킬도 직접 써서 그 결과를 원문과 대조해 보겠습니다. 마지막으로, 답마다 출처를 붙여 되짚어 주는 학습 코치도 만들어 봅니다.

[SLIDE 03] 목차

이 목표를 어떤 순서로 밟아 갈지, 목차로 한번 훑어보겠습니다. 이론 단원이 일곱 개, 그리고 마지막 줄이 실습이에요.

위의 두 줄은 스킬 이야기입니다. 스킬 파일을 원문으로 먼저 읽고요, 그다음 SOP나 프롬프트와 무엇이 다른지 가른 뒤에, 스킬이 실제로 다시 쓰이는 모습을 봅니다.

셋째부터 다섯째 줄까지는 MCP예요. 호스트, 클라이언트, 서버, 도구라는 구성을 먼저 보고, 도구를 찾아 부르는 과정과 연결 방식을 따라갑니다. 그런 다음 한 실행 안에서 스킬과 MCP가 각자 무엇을 맡는지 나눠 볼 거고요.

여섯째와 일곱째 줄은 연결의 안전 기준입니다. 에이전트가 부를 수 있는 도구를 정하는 장치를 보고, 파일 범위와 비밀값, 불완전한 응답을 어떻게 다루는지까지 짚어요. 마지막 줄의 실습 네 개에서는 도구를 직접 연결해 불러 보고, 스킬도 직접 써 보겠습니다.

[SLIDE 04] 에이전트 스킬과 SKILL.md

이번 시간에는 그 가운데 위의 두 줄, 에이전트 스킬을 다룹니다. 첫 번째 단원, 에이전트 스킬과 그 파일인 SKILL.md부터 시작하겠습니다.

쿨러 성능 저하 경보 한 건을 에이전트가 맡았다고 해 볼게요. 오늘 경보는 이렇게 처리하고, 내일 똑같은 경보는 또 다르게 처리한다면 어떨까요? 믿고 맡기기 어렵겠죠. 같은 종류의 일을 맡길 때마다 필요한 절차를 빠뜨리지 않게 하려면, 무엇을 어디에 적어 두어야 할까요?

[SLIDE 05] 목표와 도구만 받은 에이전트가 정해야 하는 것

일단 에이전트에게 목표와 도구만 줬다고 해 보겠습니다. 목표는 경보의 원인을 진단하고, 조치 후보를 비교해서, 추천안을 올리는 거예요. 그리고 에이전트 앞에는 판단 엔진과 업무 DB의 도구가 합쳐 서른 개 넘게 놓여 있습니다. 정확한 수는 첫 카드에 적어 뒀어요.

이 상태라면 에이전트가 스스로 정해야 하는 게 화면의 카드 네 장이에요.

첫 카드, 무엇부터 부를까요? 그 많은 도구 가운데 첫 호출을 골라야 합니다. 원인 진단부터 할지, 생산 오더부터 볼지, 시계열 값부터 읽을지, 고를 게 많죠.

두 번째 카드, 어떤 순서로 부를까요? 원인을 모르는 채로 조치 후보를 비교할 수는 없잖아요. 순서가 뒤바뀌면 판단에 들어가야 할 근거가 빠집니다.

세 번째 카드, 근거가 없을 때는 어떻게 할까요? 값을 읽지 못했는데 원인을 하나 골라 버리면, 그건 판단이 아니라 지어낸 답이죠.

마지막 카드, 무엇을 내고 끝낼까요? 화면에서 강조된 카드가 이 자리예요. 결과의 모양이 실행마다 다르면, 그 결과를 받아서 사람에게 승인을 받는 다음 단계가 값을 찾지 못합니다.

물론 에이전트가 매번 그럴듯하게 정할 수는 있어요. 문제는 같은 경보인데도 실행마다 과정과 결과의 모양이 달라질 수 있다는 겁니다. 업무에서는 결과가 맞는 것만큼, 같은 방법으로 했다는 걸 확인할 수 있어야 하거든요.

[SLIDE 06] 수행 방법을 지침으로 정해 실행 앞에 둔다

사실 사람의 작업에서는 이 문제를 이미 풀어 봤습니다. 작업 순서와 조건을 SOP로 적어 두고, 작업자가 그 문서를 보면서 일하게 했죠. 에이전트에게도 같은 방법을 씁니다.

그림을 볼게요. 왼쪽에서 경보가 들어오고, 그 옆이 에이전트입니다. 에이전트는 오른쪽의 지식 그래프, 시계열 DB, 업무 DB에 도구를 불러서 값을 받아요. 받은 결과를 보고 다음 호출을 정하고, 이 고리를 돌다가 아래쪽의 추천안을 내고 끝납니다.

그럼 뭐가 달라졌을까요? 맨 위에 놓인 문서 하나입니다. 부를 도구의 순서, 근거가 없을 때의 행동, 결과의 모양을 여기에 적어 두고, 에이전트가 일을 시작할 때 이걸 먼저 읽게 하는 거예요. 실제 기록을 봐도 그렇습니다. 쿨러 경보의 처리 건이 열리자, 에이전트가 가장 먼저 한 일은 이 지침 파일을 읽는 거였어요. 그때 남긴 말이 「먼저 배정된 스킬 절차를 읽고, 필요한 판단 도구를 불러오겠습니다」였습니다. 첫 도구인 원인 진단은 그 뒤에야 불렀고요.

그러니까 실행 고리 자체는 그대로예요. 돌아온 값을 읽고 그 뜻을 해석하는 일은 여전히 에이전트가 합니다. 앞 회차에서 지침은 뼈대라고 했던 게 이 뜻이죠. 달라진 건, 앞 장의 카드 네 장에 대한 답을 에이전트가 실행마다 새로 정하지 않아도 된다는 거예요. 그 고리를 어떤 순서로 돌고, 언제 멈추고, 무엇을 내고 끝낼지 기준을 주는 겁니다. 다만 읽는 지침이 생겼다고 모든 실행이 강제로 같아지는 것은 아니에요. 실제 호출과 결과가 지침을 지켰는지는 실행 기록으로 확인해야 합니다.

[SLIDE 07] 에이전트 스킬: 이름을 붙여 재사용하는 업무 지침

이렇게 적어 둔 지침을 에이전트 스킬이라고 부릅니다. 화면 첫 줄의 정의를 보세요. 특정한 일을 수행하는 방법을, 이름을 붙여 적어 둔 지침 묶음이에요.

앞 회차에서 에이전트 하나를 정하는 항목으로 목표, 도구, 업무 지침 셋을 봤습니다. 그때 업무 지침은 에이전트 스킬이라는 파일로 준다고만 하고 넘어갔죠. 이번 시간에 그 파일을 제대로 열어 봅니다.

이 정의에서 중요한 말은 「이름」입니다. 이름이 있어야 같은 일이 다시 왔을 때 그 지침을 꺼내 쓸 수 있거든요. 한 번 적고 버리는 메모가 아니라, 여러 실행에 되풀이해서 쓰는 지침이라는 뜻이죠.

둘째 줄은 파일입니다. 중심 파일은 SKILL.md, 스킬 문서입니다. 스킬 이름과 같은 폴더 안에 놓이고, 이름과 설명을 적은 머리말, 수행 방법을 적은 본문으로 이루어져요. 필요하면 참고 문서나 실행 스크립트를 같은 묶음에 둘 수도 있습니다. 이 수업의 HYD 사례에서는 중심 지침 파일부터 읽겠습니다.

셋째 줄이 실제 예예요. 쿨러 성능 저하 경보에 쓰는 HYD의 스킬은 냉각 긴급 대응이라는 뜻의 이름을 갖고 있습니다. 경보가 다시 오면 에이전트는 이 이름으로 같은 지침을 꺼내 읽습니다.

[SLIDE 08] 스킬이 놓이는 자리: 작업 폴더의 스킬 폴더

이 파일은 실제로 어디에 놓일까요? HYD에서는 에이전트가 실행할 때마다 작업 폴더를 하나씩 받습니다. 화면 왼쪽이 그 폴더의 모양이에요.

맨 위 입력 폴더부터 볼게요. 여기에는 이번에 맡은 일의 입력 파일이 있습니다. 어느 설비에서 어떤 경보가 났는지가 여기에 들어 있죠.

가운데가 스킬 폴더예요. 배정된 스킬이 폴더째 놓이는데, 폴더 이름이 곧 스킬 이름입니다. 그 안에 SKILL.md가 있고요.

맨 아래 출력 폴더에는 에이전트가 결과를 써 넣을 파일 자리가 미리 정해져 있습니다. 앞 회차에서 본 구조화 출력, 약속한 모양의 JSON이 바로 이 파일에 담겨요. 자리가 하나로 정해져 있으니까, 시스템은 늘 같은 곳에서 결과를 꺼내 형식이 맞는지 검사할 수 있죠.

쿨러 경보의 실행 기록을 봐도, 첫 줄에 배정된 스킬 하나를 작업 폴더에 넣고 실행한다고 적혀 있어요. 스킬 본문은 평소에는 포털의 데이터베이스에 보관돼 있다가, 실행이 시작될 때마다 이 자리에 파일로 써지는 거예요. 에이전트에게 주는 지시문에도 배정된 스킬의 이름과 파일 경로를 알려주는 절이 붙고요. 이렇게 입력, 지침, 결과의 자리를 알려 주면 읽고 쓸 대상을 찾기 쉬워집니다. 화면의 폴더 경로는 이 실행 환경이 택한 배치예요. 다른 제품에서는 경로가 달라도, 이번 일의 입력과 재사용할 지침과 남길 결과를 구분한다는 원리는 같습니다.

[SLIDE 09] SKILL.md 원문 ①: 머리말과 요약

이제 에이전트가 실제로 읽은 SKILL.md를 열어 보겠습니다. 화면 왼쪽이 그 앞부분이에요. 맨 위에 구분선 두 줄이 보이죠? 그 사이가 머리말입니다. 이름과 설명, 두 항목이 들어 있어요.

이름은 폴더 이름과 같은 스킬의 이름이고요, 설명은 어떤 경우에 쓰는 스킬인지를 한 줄로 적은 겁니다. 원문에는 쿨러 과열 경보에서 원인을 진단하고 냉각 조치 카드를 낸다고 적혀 있어요. 여기서 쿨러 과열 경보는, 우리가 지금까지 불러 온 쿨러 성능 저하 경보와 같은 경보입니다.

머리말 아래로 내려가면 본문이 요약 문단으로 시작해요. 경보 한 건에서 원인을 진단하고, 냉각 조치 후보를 비교해서, 추천 카드 한 장과 지는 대안을 올린다고 했죠. 여기서 말하는 추천 카드가 바로 추천안입니다.

마지막 두 줄은 꼭 짚고 갈게요. 승인 전에는 조회와 계산만 하고, 설비 명령은 사람이 승인한 뒤 시스템이 낸다고 적혀 있습니다. 스킬이 맡는 일의 범위와 그 한계를 처음부터 밝혀 둔 거예요. 에이전트가 할 일만큼, 하지 않을 일을 적어 두는 것도 지침의 몫입니다.

[SLIDE 10] 머리말의 name과 description: 스킬을 고르는 기준

머리말의 이름과 설명은 역할이 다릅니다. 이름은 어떤 스킬인지를 구별하는 식별자예요. 설명은 어떤 일에 이 스킬이 필요한지를 판단할 단서입니다.

이름은 파일과 지침 목록을 서로 연결합니다. 화면처럼 소문자와 숫자, 하이픈을 쓰고 폴더 이름과 맞춥니다. HYD의 저장 검사는 길이를 64자 이내로 제한해요. 이 규칙을 외우는 것보다, 목록에서 고른 이름이 실제로 읽을 파일과 일치해야 한다는 이유를 이해하면 됩니다.

설명에는 언제 쓰고 무엇을 내는지를 함께 적습니다. 그냥 경보 처리라고만 적으면 쿨러 경보인지 정기 정비 경보인지 가리기 어렵죠. 쿨러 경보의 원인을 진단하고 냉각 조치 카드를 낸다고 쓰면, 맡길 일과 기대하는 결과가 함께 드러납니다.

스킬을 많이 갖춘 환경에서는 먼저 이런 짧은 이름과 설명을 보여 주고, 필요한 스킬의 본문을 나중에 읽는 방식을 쓸 수 있어요. 모든 지침을 매번 한꺼번에 읽으면 관련 없는 내용까지 들어오고, 지금 필요한 절차를 찾기 어려워지기 때문입니다.

HYD에서는 그 전에 배정 과정이 하나 더 있습니다. 에이전트나 업무 단계에 지정된 스킬을 골라 작업 폴더에 넣고, 그 목록을 실행 지시문에 알려 줘요. 설명 한 줄을 적었다고 모든 에이전트가 자동으로 그 스킬을 찾아 쓰는 것은 아닙니다. 사용할 수 있게 배정하는 일과, 배정된 지침 중 지금 일에 맞는 것을 읽는 일을 구분해 두세요.

[SLIDE 11] SKILL.md 원문 ②: 절차

같은 파일에서 이번에는 절차 부분입니다. 왼쪽 원문을 보면 단계가 다섯 개 있고, 번호가 권장하는 수행 순서예요.

1단계는 입력을 읽을 자리를 가리킵니다. 작업 폴더의 입력 파일에서 설비, 경보의 종류, 경보 번호, 사건 번호를 읽으라고 했어요.

2단계는 원인 진단입니다. 부를 도구의 이름과 넘길 값이 그대로 적혀 있죠. 도구 이름 뒤의 줄표도 보세요. 원인 1순위, 고장 유형, 가이드 카드를 얻는다고, 무엇을 돌려받는지까지 적어 뒀습니다. 그리고 2단계 끝에 한 줄이 더 붙어 있어요. 진단 결과가 근거를 확인하지 못했다고 하면, 원인을 지어내지 말고 보류로 내라는 지침입니다.

3단계는 말투가 조금 다릅니다.「필요하면」업무 DB로 납기 오더와 출하 대기 로트를 확인하되,「조회만」하라고 적었어요. 꼭 해야 하는 단계인지, 어디까지 해도 되는지를 낱말로 정해 둔 거죠.

4단계는 조치 후보 비교예요. 괄호 안에 원인과 고장 유형이 다시 나옵니다. 2단계에서 돌려받은 값이 4단계에 넘길 값이 되는 거예요. 앞 회차에서 진단 결과의 두 식별자가 대안 평가의 물음에 그대로 들어간 장면을 봤죠. 원문에는 그 연결이 이렇게 적혀 있습니다. 원인을 모르면 조치 후보를 비교할 수 없다고 했던 게 바로 이 뜻이고요. 그 아래 줄에는, 추천 안이 왜 이기고 다른 안이 어디서 지는지를 한 줄씩 정리하라고 적었어요.

5단계는 판단 제출입니다. 끝에 돌려받은 번호가 판단 번호라고 적어서, 결과에 낼 값이 어디서 오는지도 정해 뒀어요.

이렇게 짚고 보면, 도구 이름이 그대로 적힌 건 2단계, 4단계, 5단계입니다. 1단계는 읽을 자리를, 3단계는 범위를 정하고요. 그러니까 에이전트는 그 많은 도구 가운데 무엇을 어떤 순서로 부를지 추측하지 않아도 됩니다. 절차가 순서와 단계마다의 범위를 알려 주거든요. 필요한 값이 다음 단계로 제대로 넘어갔는지 확인할 기준도 여기서 얻습니다.

[SLIDE 12] SKILL.md가 정하는 여섯 가지

지금까지 원문에서 본 걸 여섯 가지로 묶어 보겠습니다. 왼쪽은 SKILL.md 한 쪽을 그린 그림이고, 오른쪽은 각 구획이 맡는 일이에요.

위에서부터 갑니다. 맨 위 머리말의 설명이 호출 조건을 맡아요. 언제 쓰는 스킬인지를 정하죠. 그 아래 본문의 첫 문단은 작업 목적을 밝힙니다. 경보 한 건에서 무엇을 내는 일인지요.

가운데 절차에는 세 가지가 들어 있어요. 1단계가 입력을 읽을 자리를 가리키고요, 2단계 끝의 문장이 예외 지침을 맡고, 단계 번호가 호출의 순서가 됩니다.

맨 아래 결과 절이 출력을 정해요. 내놓을 항목의 이름을 여기에 적습니다.

그런데 기억해 둘 게 있어요. 실제 파일에는 「호출 조건」이라는 절이나 「예외」라는 절이 따로 있지 않습니다. 설명 한 줄과 절차 안의 한 문장이 그 일을 맡을 뿐이에요. 그러니까 여섯 가지는 제목이 붙은 칸이 아니라, 스킬 한 편이 빠짐없이 답해야 하는 물음입니다. 언제, 무엇을 위해 쓰고, 무엇을 받고, 막히면 어떻게 하고, 어떤 순서로 해서, 무엇을 내는가. 나중에 직접 스킬을 쓸 때 이 물음에 답이 다 있는지 확인하면 됩니다.

[SLIDE 13] 예외 지침: 근거가 없을 때 지어내지 않는다

여섯 가지 가운데 예외 지침이 실제로 쓰이는 장면을 보겠습니다. 원인 진단 도구의 응답에는 근거를 확인했는지 알려 주는 항목이 하나 있어요. 화면에 withheld라고 적힌 항목입니다.

왼쪽 칸이 근거가 확인된 응답이에요. 보류하지 않는다는 표시가 오고, 원인 1순위가 정해집니다. 쿨러 사건에서는 쿨러 핀 오염이었죠. 둘째 줄을 보면, 이때 응답에는 관측값이 허용된 나이 안에 있다는 신선도 표시도 함께 왔어요. 그러면 에이전트는 절차의 다음 단계로 넘어가요.

오른쪽 칸은 근거가 확인되지 않은 응답입니다. 판단을 보류한다는 표시가 오는 경우죠. 값이 오래됐거나, 조회한 증거가 어떤 원인과도 맞지 않을 때예요. 값이 오래됐다는 건, 앞 회차에서 본 신선도가 허용 범위를 벗어난 경우고요. 이럴 때 스킬은 원인을 고르지 말고 보류로 내라고 지시합니다. 보류로 낼 때는 앞 회차에서 본 보류의 형식대로, 상태와 이유와 증거를 담아 내죠.

왜 이 문장을 굳이 적어 둬야 할까요? 에이전트는 어떻게든 답을 만들어 내는 쪽으로 움직이기 쉽거든요. 근거가 없는데도 제일 그럴듯한 원인을 골라 다음 단계로 가 버리면, 그 뒤의 조치 비교와 추천이 전부 잘못된 출발점 위에 쌓입니다. 막혔을 때의 행동을 미리 정해 둔다는 건, 보류도 하나의 정상적인 결과로 인정한다는 뜻이에요.

[SLIDE 14] 결과 절이 정한 항목과 실제 출력

이번에는 결과입니다. 왼쪽 위가 SKILL.md의 결과 절이고, 그 아래가 에이전트가 실제로 쓴 결과 파일이에요. 결과 절이 이름을 정해 둔 다섯 항목이 파일에 그대로 들어 있는 게 보이죠?

처음 두 항목은 원인과 고장 유형입니다. 지식 그래프의 식별자를 그대로 냈어요. 쿨러 핀 오염, 그리고 쿨러 냉각 성능 상실을 가리키는 값이에요.

가운데 판단 항목에는 추천한 조치와 설명, 후보의 순위가 들어 있습니다. 추천 값은 팬 최대 운전을 가리키는 식별자고요. 이름이 skill로 시작하지만, 지금 배우는 에이전트 스킬이 아니라 조치 방법을 가리키는 값이에요. 둘은 잠시 뒤에 가릅니다. 설명에 무엇을 쓸지도 스킬 원문의 결과 절이 정해 뒀어요. 원인, 추천한 안, 그리고 다른 안이 왜 지는지를 두세 문장으로 쓰라고 했습니다. 절차 4단계에서 한 줄씩 정리하라던 그 까닭이 여기에 들어가는 거예요. 승인하는 사람이 고른 안만 보는 게 아니라, 고르지 않은 안의 까닭까지 보게 하려는 거죠.

마지막 항목은 제출한 판단에 붙은 번호예요. 그런데 쿨러 경보는 같은 스킬로 두 번 처리됐고, 이 파일은 그 둘째 건의 결과입니다. 그래서 번호도 둘째 건의 것이에요. 두 실행은 조금 뒤에 나란히 봅니다.

결과 절이 이름을 굳이 정해 두는 까닭은 받는 쪽에 있습니다. 결과를 받아서 사람에게 승인을 받는 다음 단계는, 값을 항목의 이름으로 찾아요. 이름이 실행마다 바뀌면 다음 단계가 값을 못 찾겠죠. 결과 절은 에이전트의 일과 그다음 사람의 일을 잇는 약속인 셈입니다.

이렇게 언제 쓰는지부터 무엇을 내는지까지 SKILL.md 한 편에 적어서 실행 앞에 두면, 같은 종류의 경보가 다시 와도 참조할 절차와 결과 형식을 다시 사용할 수 있습니다.

[SLIDE 15] 재사용 지침: SOP · 프롬프트와의 구분

스킬 한 편이 무엇으로 이루어지는지 원문으로 확인했으니, 두 번째 단원에서는 스킬을 비슷해 보이는 두 가지와 갈라 보겠습니다. 에이전트 스킬은 절차를 적은 문서죠. 그런데 쿨러 사건에는 이미 절차 문서가 하나 있었습니다. 팬을 최대로 올리는 SOP예요. 같은 사건에 쓰인 이 두 절차는 무엇이 다를까요? 또 방법을 매번 프롬프트로 알려 주는 것과 스킬로 남겨 두는 것은 어떻게 다를까요? 이 두 가지를 가른 다음, 스킬이 실제로 다시 쓰인 모습까지 확인하겠습니다.

[SLIDE 16] 한 사건에 쓰인 두 절차: 에이전트 스킬과 SOP

쿨러 사건 한 건을 시간 순서로 늘어놓았습니다. 왼쪽 끝이 경보가 난 때고요, 가운데를 조금 지난 자리가 운전원이 승인한 때예요. 정확한 시각은 라벨에 적어 뒀는데, 경보에서 승인까지 2분이 채 안 걸렸습니다.

왼쪽 하늘색 구획부터 볼게요. 에이전트 스킬이 이끄는 구간입니다. 아래 줄을 따라가면 SKILL.md, 에이전트, 추천안이 차례로 있죠. 에이전트가 스킬을 읽고, 조회와 계산을 해서 추천안을 냈어요. 모두 승인 전의 일입니다.

운전원이 팬 최대 운전을 승인하고 나면 오른쪽 회색 구획으로 넘어갑니다. 아래 줄의 SOP-COOL-11과 팬 100 %가 여기 있어요. 이 구간에서는 SOP가 설비에 할 일을 정했습니다. 팬 속도를 100 %로 올리는 작업이죠. 그 명령도 에이전트가 아니라, 승인을 받은 시스템이 냈고요.

이렇게 두 절차가 쓰인 때는 사람의 승인을 사이에 두고 딱 갈립니다. 승인 전에는 정보를 모으고 판단을 만드는 절차가, 승인 뒤에는 설비를 실제로 바꾸는 절차가 쓰여요. 스킬 원문의 마지막 두 줄, 승인 전에는 조회와 계산만 한다는 문장 기억나시죠? 그 문장이 바로 이 경계를 적어 둔 거였습니다.

[SLIDE 17] SOP와 에이전트 스킬: 읽는 이와 적는 내용

두 문서를 나란히 놓으면, 이 사례에서 누구에게 무엇을 알려 주는지가 분명해집니다. 왼쪽 SOP는 현장 작업자가 참고하는 표준 작업 절차예요. 설비를 다루는 순서와 안전 조건, 금지 사항을 적습니다. 화면의 팬 최대 운전 절차가 그 예입니다.

오른쪽 에이전트 스킬은 AI가 업무를 수행할 때 읽는 지침이에요. HYD의 냉각 대응 스킬에는 어떤 도구로 원인을 확인하고, 근거가 부족하면 어떻게 멈추며, 어떤 항목으로 추천안을 낼지가 적혀 있습니다.

여기서 승인 전후의 구분은 HYD가 이 업무에 정한 역할 배분입니다. SOP라는 말이 반드시 승인 뒤의 설비 조작만 뜻하는 것은 아니에요. 작업 전 안전 확인이나 정보 확인도 SOP에 들어갈 수 있습니다. 에이전트 스킬 역시 일반적으로는 다양한 업무의 지침을 담을 수 있고요.

이 수업에서 지킬 경계는 구체적입니다. 냉각 대응 에이전트는 판단을 만들고 제출하며, 설비 조작은 사람의 승인 뒤 시스템이 맡습니다. 같은 절차 문서라도 누가 읽는지, 무엇을 수행하는지, 실행 권한은 어디에 있는지를 함께 봐야 합니다. 문서에 하라고 썼다는 사실만으로 그 권한이 생기지는 않습니다.

[SLIDE 18] 「Skill」의 두 쓰임: 조치 방법과 에이전트 스킬

여기서 이름 때문에 헷갈리기 쉬운 걸 하나 정리하고 가겠습니다. 지식 그래프에서 조치 방법의 클래스 이름이 Skill이었죠. 앞의 결과 파일에서 본 추천 값도 바로 그 클래스의 식별자였고요.

표의 가운데 열이 조치 방법입니다. SOP 한 편을 지식 그래프로 옮긴 노드예요. 작업 단계, 승인 역할, 근거 매뉴얼과 관계로 이어져 있고, 조치 후보로 추천되고 승인됩니다.

오른쪽 열이 이번 회차에 배우는 에이전트 스킬이에요. 에이전트가 읽는 지침 파일이고, 도구를 부르는 순서와 결과의 모양을 담고 있죠. 판단을 수행할 때 읽힙니다.

이 HYD 스킬 본문은 특정 조치 방법 노드를 직접 가리키지 않아요. 에이전트 스킬 본문에는 팬 최대 운전이라는 조치 방법이 적혀 있지 않습니다. 조치 후보는 도구가 지식 그래프에서 읽어서 돌려줄 뿐이에요. 그래서 이 과정에서는 지식 그래프 쪽을 조치 방법, 지침 파일 쪽을 에이전트 스킬, 이렇게 나눠서 부르겠습니다.

[SLIDE 19] 스킬에 적힌 것과 입력 · 도구에서 얻는 것

방금 스킬 본문에 팬 최대 운전이 적혀 있지 않다고 했죠. 사실 HYD-01이라는 설비 이름도, 쿨러 핀 오염이라는 원인도 안 적혀 있습니다. 그럼 이런 정보는 다 어디서 올까요?

표의 첫 줄이 스킬에 적힌 것입니다. 도구를 부르는 순서, 보류의 조건, 결과 항목의 이름. 그러니까 방법만 있어요.

둘째 줄, 설비와 경보 번호는 작업 폴더의 입력 파일에서 읽습니다. 셋째 줄, 원인과 조치 후보는 도구가 지식 그래프에서 읽어 주고요. 쿨러 핀 오염이라는 원인, 그리고 팬 최대 운전을 비롯한 네 가지 후보가 여기서 옵니다. 넷째 줄의 지금 값, 냉각 효율이 지금 얼마인지, 납기까지 몇 시간 남았는지 같은 값은 도구가 조회하는 그 시점에 가져오죠.

이렇게 나눠 두면 같은 업무 규칙을 적용할 수 있는 다른 사건에도 지침을 다시 쓸 수 있어요. 설비나 경보 번호가 달라졌다는 이유만으로 절차를 다시 쓸 필요가 줄어드는 거죠. 다만 대상 설비

가 같은 진단을 지원하는지, 필요한 도구와 규칙이 맞는지는 확인해야 합니다. 앞 회차에서 지식과 현재값을 섞지 말라고 했죠. 이번 HYD 스킬은 사건별 원인과 현재 수치를 직접 적어 두지 않습니다. 만약 스킬에 냉각 효율 값을 적어 뒀다면 어떨까요? 다음 경보에서는 그 값이 이미 묵은 값인데도 판단에 들어가겠죠. 사건마다 달라지는 것은 스킬에 적지 않는다, 이게 재사용의 조건이에요.

[SLIDE 20] 같은 스킬, 다른 경보: 두 번의 실행

그 재사용이 실제로 일어난 모습입니다. 왼쪽의 스킬 한 편이, 오른쪽의 서로 다른 쿨러 경보 두 건에 쓰였어요.

오른쪽 위의 첫째 경보와 그 아래 둘째 경보는 번호가 다른 별개의 건입니다. 두 실행 모두 같은 스킬을 읽고 같은 절차를 따랐어요. 그리고 저마다 판단을 하나씩 제출했죠. 판단 번호도 서로 다릅니다.

다만 같은 스킬을 따랐다고 해서 두 실행이 한 글자까지 똑같았던 건 아니에요. 앞 회차에서 봤듯이, 둘째 실행은 지침에 없던 조치 효과 예측을 더 불렀죠. 스킬이 제공하는 건 함께 참고할 절차의 뼈대와 결과 형식입니다. 실제로 지켰는지는 두 실행의 호출과 출력으로 따로 확인합니다.

맨 아래 흐린 줄은 아직 오지 않은 경보예요. 같은 종류의 경보가 또 오면, 같은 스킬이 또 쓰입니다.

한번 정리한 지침을 여러 실행이 함께 쓸 수 있으니, 매번 설명할 수고가 줄어듭니다. 그런데 반대도 마찬가지예요. 지침에 잘못이 있으면 그 잘못도 모든 실행에 따라갑니다. 그래서 스킬은 한 번 쓰고 끝내는 게 아니라, 실행 기록과 대조하면서 고쳐 가는 자산이에요.

[SLIDE 21] 한 번의 프롬프트와 재사용하는 지침

방법을 그때그때 프롬프트에 적어 주는 방식과, 이름을 붙여 관리하는 스킬을 비교해 보겠습니다. 둘 다 에이전트에게 읽힐 수 있는 지침이에요. 차이는 반복해서 쓸 방법을 어떻게 보관하고 적용하느냐에 있습니다.

왼쪽은 이번 요청에만 필요한 조건을 직접 적는 경우입니다. 오늘 받은 회의록 한 편을 정리해 달라는 요청처럼, 입력과 목적이 이번 일에 묶여 있죠. 같은 절차를 매번 복사해서 붙이면 수정한 판과 옛 판이 섞이거나 중요한 예외가 빠질 수 있습니다.

오른쪽은 반복되는 절차를 스킬로 관리하는 경우예요. 회의록이 바뀌어도 담당자와 기한이 없으면 미정으로 남긴다는 규칙은 같죠. 그 공통 방법을 이름 있는 지침으로 보관하고, 요청에는 이번 회의록과 추가 조건을 주면 됩니다.

프롬프트도 템플릿으로 저장해서 다시 쓸 수 있습니다. 스킬만 재사용할 수 있다는 뜻은 아니에요. 스킬의 가치는 관련 지침과 자료를 한 묶음으로 관리하고, 필요한 업무에서 찾아 쓰도록 연결하는 데 있습니다.

HYD에서는 보관된 스킬을 실행 준비 때 작업 폴더에 써 줍니다. 보관본을 수정하면 이후 준비되는 실행에서 새 내용을 받는 방식이에요. 이미 진행 중인 모든 실행이 즉시 바뀐다고 생각하면 안 됩니다. 어떤 실행에 어느 지침이 제공됐는지 남겨야, 수정 전후 결과를 비교할 수 있습니다.

[SLIDE 22] 같은 틀의 세 스킬과 그 스킬을 붙인 에이전트

이렇게 이름을 붙여 남긴 HYD의 기본 시나리오 스킬 세 개를 보면, 저마다 에이전트 하나에 붙어 있습니다. 표의 첫 칸이 그 에이전트예요. 쿨러 경보를 맡는 냉각 긴급 대응, 정기 정비 시기를 정하는 정기 정비 계획, 예비품 발주를 맡는 예비품 구매입니다.

가운데 칸에는 각 스킬의 이름과 설명이 있어요. 둘째 줄의 설명을 위에서부터 이어 읽어 볼게요. 쿨러 과열 경보에는 원인 진단과 냉각 조치 카드, 정기 정비가 돌아오면 언제 정비할지 카드, 예비품 재고가 기준을 벗어나면 발주 카드. 셋 다 앞에서 본 설명의 꼴 그대로, 무엇이 오면 쓰고 무엇을 내는지를 한 줄에 담았죠. 세 스킬 모두 머리말, 절차, 결과로 이루어져 있어서 틀도 같습니다. 다른 건 절차에 적힌 업무, 그리고 그 절차가 부르는 도구예요.

오른쪽 칸은 각 에이전트에 연결한 서버입니다. 스킬의 절차가 부를 도구가 있는 곳이죠. 여기서 서버는 일단, 도구를 모아서 내주는 프로그램이라고 생각하시면 됩니다. 칸의 첫 줄, 지식 그래프 서버와 판단 엔진 서버는 세 에이전트가 똑같이 연결했어요. 다른 건 둘째 줄의 업무 DB 쪽입니다. 냉각 긴급 대응은 업무 DB 전체에 닿는 서버를, 정기 정비 계획은 정비 쪽 서버를, 예비품 구매는 구매 쪽 서버를 연결했어요. 왜 이렇게 나눴는지는 안전 기준을 다룰 때 다시 보겠습니다.

하나 기억해 둘 게 있어요. 스킬은 만들어 두기만 해서는 쓰이지 않습니다. HYD에서는 에이전트의 배정 목록이나 업무 단계의 지정에 들어 있어야 해당 실행에 제공됩니다. 그래서 새 업무를 맡길 때는 세 가지를 해야 합니다. 같은 틀에 그 업무의 절차를 적고, 그 일을 맡을 에이전트에 붙이고, 절차가 부를 도구를 연결하는 거죠.

[SLIDE 23] 스킬을 쓸 때 지키는 네 가지

이제 스킬을 직접 쓸 때 지킬 것을 네 가지로 줄여 보겠습니다. 실습에서 회의록 스킬을 쓸 때 이 카드 네 장을 그대로 씁니다.

첫 카드, 설명에는 어떤 요청 한 건에서 무엇을 내는 스킬인지를 한 줄로 밝힙니다. 사람이 배정할 때도, 에이전트가 적용할 절차를 찾을 때도 이 설명을 단서로 쓰니까요.

두 번째, 절차의 도구 이름은 에이전트에 연결된 도구의 실제 이름을 그대로 옮겨 적습니다. 그럴듯한 이름을 지어내면, 에이전트는 없는 도구를 찾다가 멈춰요.

세 번째, 근거나 남는 후보가 없을 때 어떻게 할지 정해 둡니다. 지어내지 말고, 없으면 없는 그대로 내게 하는 거예요. 예를 들어 규칙으로 후보를 다 빼고 나니 남는 안이 없다면, 추천을 억지로 만들지 말고 빈 목록과 함께 안마다 빠진 이유를 내게 하는 식이죠.

네 번째, 결과 항목의 이름을 정해 두고 바꾸지 않습니다. 다음 단계가 그 이름으로 값을 읽으니까요.

이 네 가지는 스킬이 SOP나 프롬프트와 다른 바로 그 지점에서 나옵니다. SOP는 사람이 읽지만 스킬은 에이전트가 읽죠. 모호한 문장을 모델이 그럴듯하게 해석하면 의도와 다른 도구를 고를 수 있어요. 재사용되는 지침의 모호함은 다음 실행에도 이어질 수 있고요. 그래서 스킬은 짧더라도 고르는 기준, 부를 이름, 막혔을 때의 행동, 내놓을 이름, 이 네 가지가 분명해야 합니다.

다음 시간에는 스킬이 부르는 도구 쪽으로 넘어갑니다. 절차에 이름만 적힌 그 도구가 실제로 어디에 있고, 에이전트가 어떤 표준으로 그 도구를 부르는지 보겠습니다.

[이론 02] AI가 여러 시스템의 기능을 사용하는 원리 이해하기

[SLIDE 24] MCP의 구성: 호스트 · 클라이언트 · 서버 · 도구

이번 시간에는 스킬이 부르는 도구 쪽을 다룹니다. 도구를 잇는 표준인 MCP의 구성부터, 도구를 찾아 부르는 과정과 연결 방식, 그리고 한 실행 안에서 스킬과 MCP가 나눠 맡는 몫까지 봅니다.

앞 시간에 본 에이전트 스킬에는 부를 도구의 이름이 적혀 있었죠. 그런데 이름을 적었다고 해서 도구가 실행되지는 않아요. 그 도구는 어디에 있고, 에이전트는 어떤 약속을 따라서 그걸 부를까요? 세 번째 단원에서는 그 약속, MCP가 무엇으로 이루어져 있는지 보겠습니다.

[SLIDE 25] 스킬의 절차가 가리키는 것: 스킬 밖의 도구

스킬 절차의 2단계 한 줄을 화면 가운데에 크게 띄웠습니다. 이 한 줄 안에 이름이 세 개 들어 있어요.

맨 앞은 도구를 내주는 프로그램의 이름입니다. 판단 엔진을 앞에 두고 도구를 내주는 프로그램이에요. 원인 진단은 실제로 여기서 실행됩니다.

가운데는 그 프로그램이 내주는 도구, 원인 진단의 이름이고요. 괄호 안은 도구에 넘길 값의 이름이에요. 설비, 경보의 종류, 경보 번호죠. 이 세 값은 바로 앞의 1단계에서 입력 파일로 읽은 것들입니다. 입력에서 읽은 걸 그대로 도구에 넘기는 거예요.

이렇게 세 이름이 저마다 하나씩 답하고 있습니다. 어느 프로그램에 보낼지, 무엇을 실행할지, 무엇에 대해 실행할지요. 결국 스킬은 이 이름들을 적어 둔 글일 뿐입니다. 원인을 진단하는 계산은 스킬 안에 없어요. 스킬 밖의 프로그램에서 일어납니다.

[SLIDE 26] 시스템마다 따로 잇는 방식과 표준을 거치는 방식

바깥 시스템과 연결하는 방법이 제각각이면 어떤 일이 생길까요? 화면 왼쪽에는 애플리케이션 셋과 시스템 셋이 있습니다. 각각의 조합마다 별도 규격에 맞춘 코드를 만들면, 관리할 조합은 아홉 가지가 됩니다.

오른쪽은 양쪽이 공통 규약을 따르는 방식입니다. 애플리케이션 쪽은 공통 규약으로 요청을 보내고 결과를 읽는 기능을 갖춥니다. 시스템 쪽은 자기 기능을 그 규약에 맞게 내주는 서버를 마련하고요. 화면의 셋 더하기 셋은 이렇게 표준에 맞춰 구현할 쪽을 세는 개념도입니다.

이 숫자를 실제 연결선이나 접속 횟수로 읽으면 안 됩니다. 애플리케이션 하나가 서버 셋을 쓰면 서버마다 연결이 필요합니다. 또 가운데 파란 띠는 모든 요청이 지나가는 중앙 서버가 아니라, 양쪽이 함께 따르는 약속을 나타냅니다.

왜 이런 약속이 도움이 될까요? 시스템 쪽에서 기능을 어떻게 공개하는지 일정해지면, 애플리케이션마다 같은 연결 작업을 처음부터 반복할 일이 줄어듭니다. 반대로 업무 시스템의 데이터 구조와 권한은 여전히 다르니, 그 차이는 각 서버가 맞춰 줘야 합니다. 표준이 줄이는 것은 반복되는 연결 형식의 차이이고, 업무의 의미까지 자동으로 같게 만드는 것은 아니에요.

[SLIDE 27] MCP: AI 애플리케이션과 외부 도구를 잇는 개방형 프로토콜

그 표준이 MCP, Model Context Protocol입니다. 화면 첫 줄의 정의를 볼게요. AI 애플리케이션이 외부의 도구와 데이터에 연결하는 방식을 하나로 정한 개방형 프로토콜이에요.

개방형이라는 말은, 누구나 이 규약에 맞춰 만들 수 있다는 뜻입니다. 그래서 서로 다른 곳에서 만든 애플리케이션과 서버도 공통 규약을 바탕으로 연결할 수 있습니다. 실제 연결에는 지원하는 규약의 판, 연결 방식, 필요한 기능과 인증 조건이 맞아야 해요. HYD에도 그런 예가 있어요. 지식 그래프 서버는 HYD가 만든 게 아니라, 바깥에 공개된 서버를 그대로 가져다 붙인 겁니다. 판단 엔진 서버와 업무 DB 서버는 HYD가 직접 만들었고요. 만든 곳은 달라도, 에이전트 쪽에서는 똑같은 방식으로 연결합니다.

둘째 줄은 메시지 형식이에요. MCP는 JSON-RPC 2.0이라는 형식으로 메시지를 주고받습니다. 원격으로 기능을 부르는 약속을 JSON으로 적은 형식이죠. 요청에는 부를 기능의 이름과 넘길 값이 담기고, 응답에는 결과나 오류가 담겨요.

셋째 줄은 비유입니다. 규격이 같은 단자를 떠올려 보세요. 만든 회사가 달라도 규격이 같으면 꽂히죠. 그런데 다른 점도 있어요. MCP는 연결한 다음에 쓸 수 있는 기능을 맞추고, 서버가 무엇을 내주는지 물어서 알아내는 절차가 있습니다. 이 절차가 뒤에서 볼 도구 목록의 발견이에요.

[SLIDE 28] MCP의 네 구성요소: 호스트 · 클라이언트 · 서버 · 도구

이번에는 MCP로 도구를 호출하는 데 필요한 네 역할을 보겠습니다. 호스트, 클라이언트, 서버, 도구. 그림 왼쪽부터 하나씩 볼게요.

왼쪽의 큰 하늘색 구획이 호스트입니다. 라벨에 적은 대로, HYD에서는 에이전트 프로그램이 이 자리에 있어요. 그 안에 모델과 클라이언트가 들어 있고, 모델은 LLM입니다.

호스트 안에 작은 상자가 세 개 있죠? 이게 클라이언트입니다. 클라이언트 하나는 서버 하나와 일대일로 연결돼요. 서버가 셋이니까 클라이언트도 셋입니다.

가운데 흰 상자들이 서버예요. 위에서부터 지식 그래프 서버, 판단 엔진 서버, 업무 DB 서버입니다. 서버는 오른쪽 끝에 있는 시스템의 기능을 도구로 내줘요. 상자 안의 톱니바퀴가 바로 도구고요.

그림은 가로로 한 줄씩 읽으면 됩니다. 맨 윗줄은 클라이언트 하나, 지식 그래프 서버, 그 뒤의 지식 그래프가 한 줄로 이어져 있죠. 가운뎃줄은 판단 엔진까지, 아랫줄은 업무 DB까지 같은 모양이에요. 이 그림은 각 서버의 주된 대상을 한 줄씩 놓은 것입니다. 서버 하나가 여러 시스템의 값을 모아 주는 구성도 가능합니다.

그래서 에이전트마다 연결할 서버를 고르면, 클라이언트도 딱 그 서버 수만큼만 생깁니다. 서버를 하나 빼면 그 줄이 통째로 사라지는 거죠.

[SLIDE 29] 네 구성요소의 정의와 HYD의 대응

그림으로 자리를 봤으니, 네 구성요소의 정의를 정리하고 HYD에서는 그 자리에 무엇이 있는지 짝지어 보겠습니다. 정의는 화면 가운데 칸의 첫 줄에, HYD의 대응은 둘째 줄에 적어 뒀어요.

첫 줄, 호스트는 모델을 가지고 사용자의 일을 수행하는 애플리케이션이고, 클라이언트를 만들고 관리합니다. HYD에서는 처리 건마다 띄워지는 에이전트 프로그램이 이 자리에 있어요. 모델을 부르고, 연결할 서버마다 클라이언트를 두는 쪽이죠. 이 실행 사례에서는 작업 폴더를 마련하고 프로그램을 띄우는 워커와, 그 안에서 모델과 MCP 연결을 관리하는 호스트를 구분합니다.

둘째 줄, 클라이언트입니다. HYD에서는 실행할 때마다 작업 폴더에 연결할 서버의 설정 파일이 놓이고요, 호스트가 거기 적힌 서버마다 클라이언트를 하나씩 둡니다. 그래서 그 에이전트에 연결한 서버 수만큼 생기는 거예요.

셋째 줄, 서버는 지식 그래프, 판단 엔진, 업무 DB 앞에 놓인 프로그램들입니다.

넷째 줄, 도구는 서버가 내주는, 호출할 수 있는 기능이에요. 이름과 설명과 입력 스키마를 갖고 있습니다. 원인 진단, 조치 후보 비교, 생산 오더 조회 같은 것들이죠.

HYD에는 클라이언트를 가진 쪽이 하나 더 있어요. 포털의 도구 시험 화면입니다. 브라우저 화면의 요청을 받아 포털 백엔드가 MCP 클라이언트로 서버에 연결하고 목록과 결과를 가져옵니다. 실습에서 서버를 등록하고 시험해 볼 때 쓰는 곳이 바로 이 화면입니다.

서버는 도구 외에도 읽을 자료나 재사용할 프롬프트를 제공할 수 있습니다. 이번 HYD 사례에서는 호출 가능한 도구를 중심으로 보는 거예요. 그리고 모델과 호스트는 꼭 구별해 두세요. 모델은 무엇을 부를지 고르는 쪽이고, 호스트는 그 선택을 받아서 실제 연결을 관리하는 애플리케이션이에요. 이 구별은 뒤에서 안전 기준을 다룰 때 다시 씁니다.

[SLIDE 30] HYD의 MCP 서버 다섯

이 교재가 확인한 HYD의 기본 등록 MCP 서버는 다섯 개입니다. 표를 보면 서버마다 닿는 시스템과 내주는 도구가 다르죠.

첫 줄은 지식 그래프 서버예요. 도구 가운데 그래프의 구조를 읽는 것과 조회문을 실행하는 것, 딱 이 둘만 허용해 뒀습니다.

둘째 줄, 강조된 게 판단 엔진 서버입니다. 원인 진단, 조치 후보 비교, 시계열 조회 같은 도구 19개를 내줘요. 쿨러 사건에서 가장 많이 불린 서버죠. 닿는 시스템 칸을 보면 판단 엔진과 시계열 DB가 같이 적혀 있어요. 판단에 필요한 측정값까지 이 서버가 직접 읽기 때문에, 시계열 조회 도구가 여기에 있는 거예요.

셋째 줄은 업무 DB 전체를 내주는 서버로, 도구가 14개예요. 넷째와 다섯째 줄은 업무 DB 가운데 정비 쪽과 구매 쪽만 내주는 서버고요. 도구가 각각 네 개, 세 개입니다. 정비 쪽에는 정비 이력과 정기 정비 상태 같은 도구가, 구매 쪽에는 예비품 재고와 부품 견적 같은 도구가 들어 있어요.

쿨러 경보를 맡은 에이전트가 쓴 건 위의 세 서버예요. 스킬 2단계의 원인 진단도 이 가운데 판단 엔진 서버에 있습니다. 이렇게 스킬에 적힌 도구는 이 서버들 가운데 어딘가에 있고, 호스트는 서버마다 둔 클라이언트를 거쳐서, MCP의 약속대로 그 도구를 부릅니다.

[SLIDE 31] 도구의 발견 · 호출 · 연결 방식

네 구성요소가 무엇인지 알았으니, 네 번째 단원에서는 그 사이로 메시지가 어떻게 오가는지 따라가 보겠습니다. 호스트는 서버에 어떤 도구가 있는지 어떻게 알까요? 도구를 부를 때는 무엇을 넘기고, 무엇을 돌려받을까요? 그리고 클라이언트와 서버는 어떤 길로 이어질까요?

[SLIDE 32] 도구 목록의 발견과 호출: tools/list와 tools/call

도구를 쓰기까지의 과정을 왼쪽부터 세 단계로 보겠습니다. 첫째는 연결을 시작하는 초기화, 둘째는 도구를 알아보는 발견, 셋째는 실제로 도구를 부르는 호출입니다. 각 단계에서 클라이언트와 서버가 주고받는 내용이 달라요.

첫 단계에서는 초기화 요청을 주고받으며 규약의 판과 지원 기능을 확인합니다. 서버가 도구를 제공하는지도 이때 알려 줘요. 초기화가 끝났다는 알림까지 보내면 정상적인 요청을 주고받을 준비가 됩니다. 서로 지원하는 규약 판이 맞아야 그다음 요청을 주고받을 수 있는 거죠.

그다음은 도구 목록 요청입니다. 이름과 설명, 입력 스키마를 받아 어떤 기능에 무엇을 넘겨야 하는지 알아봅니다. 목록이 크면 나눠 받아 이어 읽을 수도 있어요. 여기서는 도구 목록을 요청하는 표준 이름인 tools/list를 함께 기억해 두면 됩니다.

필요한 도구를 고른 다음에는 도구 이름과 입력 값을 담아 호출합니다. 이때의 표준 이름이 tools/call이에요. 목록을 알아보는 요청과 업무 기능을 실행하는 요청이 서로 다릅니다.

HYD 포털의 연결 검사는 초기화하고 목록을 실제로 받아 보는 데까지 합니다. 그래서 통과했다는 말은 그 설정으로 대화하고 목록을 읽었다는 뜻이에요. 특정 도구에 업무 입력을 넣었을 때 성공하는지, 결과가 업무상 옳은지는 다음 호출에서 확인해야 합니다. 목록 조회가 모든 호출 직전의 필수 규칙인 것은 아니지만, 지금 사용할 도구의 이름과 입력 형식을 아는 데 필요한 발견 과정입니다.

[SLIDE 33] 도구 목록을 연결할 때 묻는 까닭

그런데 굳이 연결할 때마다 서버에 목록을 물어야 할까요? 호스트에 도구를 미리 적어 두면 더 간단해 보이는데 말이죠.

왼쪽처럼 도구를 호스트에 미리 적어 두면 어떻게 될까요? 서버의 도구가 바뀔 때마다 호스트도 고쳐야 합니다. 고치는 걸 잊으면 적어 둔 내용과 실제 서버가 어긋나고요.

오른쪽처럼 연결할 때 서버에 물으면, 서버가 지금 내주는 도구를 그대로 받아요. 이름과 설명과 입력 스키마가 목록에 함께 오니까, 도구 정보를 호스트 코드에 일일이 옮겨 적는 수고를 줄일 수 있습니다.

실제 예가 있어요. 판단 엔진 서버에는 나중에 구매 판단을 위한 도구 셋이 더해졌습니다. 그래도 에이전트 쪽 설정은 서버 주소 그대로예요. 연결할 때마다 새 목록을 받아 오니까요. 포털의 도구 시험 화면도 같은 생각을 따릅니다. 도구를 부르기 직전에 서버에서 목록을 다시 받아서, 서버가 지금 내주는 그대로를 기준으로 삼아요. 목록이 바뀌면 새 기능이 생긴 것을 알아볼 수 있죠. 다만 이름이 보인다고 바로 허용되는 것은 아닙니다. 지원하지 않는 기능이 있거나 권한 범위가 바뀌면 설정과 허용 목록도 검토해야 해요.

[SLIDE 34] 도구 정의 한 건: part_quotes

그 목록에 든 항목 하나를 그대로 옮겨 왔습니다. 화면 왼쪽이에요. 구매 쪽 업무 DB 서버가 내주는 부품 견적 조회 도구입니다.

맨 위가 도구의 이름이에요. 호출할 때 이 이름을 씁니다.

그 아래가 설명이고요. 부품별로 공급사의 단가, 불량률, 납기, 승인 여부를 읽는다고 적혀 있죠.

가운데 입력 스키마는 넘길 값을 정합니다. 부품 번호 하나를 글자로 받고, 생략하면 이 서버 구현에서는 펌프 축 씰 키트를 기본으로 씁니다. 값의 이름과 형, 그리고 비웠을 때 쓸 값까지 적혀 있으니, 모델이 무엇을 어떤 모양으로 넣을지 판단할 단서가 됩니다. 이 정의는 JSON Schema라는 표준 형식으로 적습니다. 스키마의 기본값 표시는 안내이고, 검사기가 빠진 값을 자동으로 채운다는 뜻은 아니에요. 실제 기본값 처리는 이 도구의 구현에서 합니다.

맨 아래에는 이 도구가 읽기만 한다는 표시가 붙어 있어요.

오른쪽에서 강조한 게 설명이죠. 모델이 이 도구를 부를지 말지 정할 때 읽는 게 바로 이 설명과 입력 스키마거든요. 그러니까 도구의 설명은 사람을 위한 주석에 그치지 않습니다. 설명이 모호하면 모델이 엉뚱한 도구를 고르거나, 필요한 도구를 그냥 지나쳐요. 스킬의 설명이 스킬을 고르는 기준이었던 것처럼, 도구의 설명은 도구를 고르는 기준입니다.

[SLIDE 35] 호출 한 건의 요청: 도구의 전체 이름과 입력

도구를 실제로 부를 때는 어떤 요청이 갈까요? 둘째 쿨러 경보의 실행 기록에서, 원인 진단을 부른 요청을 원래 모양 그대로 꺼냈어요.

첫 줄이 도구의 전체 이름입니다. 앞에 MCP 표시, 가운데에 서버 이름, 끝에 도구 이름이 있고, 이 실행 도구에서는 밑줄 두 개로 이어 표시했어요. MCP가 모든 제품에 이 이름 모양을 요구하는 것은 아닙니다. 판단 엔진 서버의 원인 진단 도구라는 뜻이죠.

그 아래가 입력입니다. 원인 진단 도구도 앞의 부품 견적 도구처럼 입력 스키마를 갖고 있고, 거기 정해 둔 항목이 설비, 경보의 종류, 경보 번호 셋이에요. 그 세 항목을 이름과 형에 맞춰 채운 JSON입니다.

앞에서 스킬의 한 줄에 이름이 셋 들어 있었죠. 서버 이름, 도구 이름, 넘길 값. 실제 요청에서는 앞의 두 이름이 전체 이름 하나로 합쳐지고, 넘길 값은 입력 JSON이 된 겁니다.

전체 이름에 서버 이름을 붙이는 데는 까닭이 있습니다. 같은 이름의 도구가 두 서버에 있을 수 있거든요. 앞에서 본 부품 견적 도구가 그래요. 업무 DB 전체 서버에도 있고, 구매 쪽 서버에도 있습니다. 앞에 서버 이름이 붙어 있으니 어느 서버의 것인지 헷갈리지 않죠.

그리고 입력 스키마는 넘길 값의 모양을 정한 계약이기도 합니다. 포털에서 도구를 시험해 볼 때, 입력이 스키마를 어기면 서버에 보내기도 전에 거절해요. 설비 칸에 숫자를 넣었거나 필요한 항목을 빠뜨린 요청은 서버까지 가지 않습니다. 형식이 맞지 않는 요청을 먼저 걸러 주는 겁니다. 하지만 문자열이라는 형식이 맞아도 다른 설비의 번호를 넣을 수 있죠. 스키마 검사를 통과한 것과 업무 대상을 정확히 고른 것은 따로 확인해야 합니다.

[SLIDE 36] 호출 한 건의 응답: mes_orders

이번에는 생산 오더 조회에서 돌아온 내용입니다. 화면은 HYD의 업무 DB 도구가 만든 업무 결과 본문이에요. 앞서 본 호출 이름의 모양처럼, 이 항목 이름도 HYD 사례와 MCP 공통 규약을 구분해서 읽어야 합니다.

맨 위 성공 표시는 이 업무 조회를 처리했다는 뜻입니다. 그 아래에는 어느 업무 시스템에서 가져왔는지와 기준 시각이 있어요. 판단용 값에는 오더 번호, 납기까지 남은 시간, 남은 수량이 들어 있고, 원래 행에는 그 값을 다시 확인할 자료가 담깁니다. 모든 서버가 이 항목 이름을 쓰는 것은 아닙니다.

바깥 MCP 응답도 함께 생각해 볼게요. 요청 자체를 처리하지 못한 프로토콜 오류가 있고, 요청은 전달됐지만 도구 실행에 실패해 오류 표시가 담긴 결과가 올 수도 있습니다. 그러니 결과가 돌아왔다는 사실만으로 업무가 성공했다고 판단하면 안 돼요. 포털 구현도 연결·프로토콜 상태와 도구 오류 표시를 따로 다룹니다.

업무 결과를 얻은 뒤에도 확인은 남습니다. 어느 설비의 어떤 조건을 조회했는지, 기록의 기준 시각이 판단에 충분히 가까운지 봐야 하죠. 성공 표시는 이번 요청의 처리 상태를 알려 줍니다. 답이 실제 업무 판단에 맞는지는 대상과 시각과 근거를 함께 대조해서 정합니다.

[SLIDE 37] 도구 호출 한 번이 지나는 길

요청과 응답을 봤으니, 호출 한 번이 처음부터 끝까지 지나는 길을 이어 보겠습니다. 화면에 칸이 다섯 개 있죠. 칸 사이에 적힌 작은 글자는 다음 칸으로 넘어갈 때 하는 일이에요. 요청, 확인, tools/call, 조회. 왼쪽부터 갈게요.

첫 칸, 모델이 하는 일은 어느 도구를 어떤 값으로 부르겠다는 요청을 내는 데까지입니다.

둘째 칸의 호스트가 그 요청을 받아서, 이 에이전트가 부를 수 있도록 허용된 도구인지 확인합니다. 허용된 요청이면 셋째 칸의 클라이언트가 tools/call로 서버에 전달하고요.

넷째 칸의 서버는 도구를 실행해서 마지막 칸의 시스템에서 값을 읽고, 응답을 만듭니다. 응답은 같은 길을 거슬러서 모델에게 돌아와요.

앞 회차에서 에이전트는 데이터베이스에 직접 접속하지 않는다고 했죠. 그 말이 이 길에서는 이렇게 보입니다. 모델은 맨 왼쪽 칸에, 데이터베이스는 맨 오른쪽 칸에 있고, 이 HYD 도구 호출 경로에서는 그 사이의 세 역할을 거칩니다. 어떤 프로그램이든 반드시 이 구조만 쓴다는 뜻은 아니에요.

이 길에서 강조된 칸이 호스트입니다. 모델과 실행 사이에 호스트가 서 있죠. 그래서 모델이 무엇을 요청하든, 실제로 실행될 도구를 호스트에서 가를 수 있어요. 바로 이 자리에, 부를 수 있는 도구를 정해 둔 허용 목록이 놓입니다.

[SLIDE 38] 연결 방식 두 가지: stdio와 Streamable HTTP

메시지가 오가는 모양은 봤으니, 남은 건 클라이언트와 서버를 잇는 길입니다. 여기서는 표준 전송 방식 두 가지를 보겠습니다. 화면 왼쪽이 stdio, 오른쪽이 Streamable HTTP예요.

stdio는 표준 입력과 표준 출력을 뜻합니다. 호스트가 서버 프로그램을 자기 아래 하위 프로세스로 직접 띄워요. 그리고 그 프로세스의 표준 입력으로 요청을 넣고, 표준 출력으로 응답을 받습니다. 왼쪽 아래 적힌 것처럼, 서버가 이를 띄운 클라이언트와 같은 실행 환경에서 돌아요. 그 서버가 접속하는 DB까지 같은 PC에 있어야 한다는 뜻은 아닙니다.

Streamable HTTP는 서버가 먼저 떠 있는 방식입니다. 클라이언트가 그 서버의 주소로 접속해서, HTTP 요청으로 메시지를 주고받아요. 그래서 오른쪽 그림처럼 여러 클라이언트가 한 서버를 함께 쓸 수 있습니다.

어느 쪽을 고를지는 서버를 어디에 두고 누가 쓰느냐에 달려 있어요. 내 PC에서 나만 쓰는 도구라면 stdio가 간단하고요, 여러 에이전트가 함께 쓰는 공용 서버에는 HTTP 방식을 사용할 수 있습니다. HTTP 서버를 같은 PC에서 실행하는 구성도 가능해요. 어느 방식이든 실제 데이터 접근 권한은 별도로 정합니다. HYD는 두 방식을 다 써요. 다섯 서버 가운데 지식 그래프 서버 하나만 stdio이고, 나머지 넷은 미리 떠 있는 HTTP 서버입니다.

[SLIDE 39] 서버 설정 두 건: 명령으로 띄우는 서버와 주소로 접속하는 서버

두 방식은 설정만 봐도 바로 구별됩니다. 화면 왼쪽에 HYD의 실제 서버 설정 두 건을 옮겼어요.

위쪽 지식 그래프 서버는 실행할 명령과 인자로 적혀 있습니다. 호스트가 이 명령으로 서버를 직접 띄우는 거죠. stdio 방식이에요.

아래쪽 판단 엔진 서버는 주소로 적혀 있어요. 이미 떠 있는 서버에 접속하는 Streamable HTTP 방식입니다. 실제 설정에는 지식 그래프 서버의 접속 계정과 비밀번호 칸도 있는데, 화면에서는 뺐어요. 그런 비밀값을 설정에서 어떻게 다루는지는 안전 기준에서 따로 봅니다.

앞에서 stdio 서버는 호스트와 같은 PC에서 돈다고 했죠. 그게 실제로 문제가 된 기록이 있어요. 이 교재가 참고한 기록에서는 포털 백엔드 쪽에서 지식 그래프 서버를 시험해 보려다가, 띄울 명령을 찾지 못해서 실패했습니다. 이 서버는 에이전트 프로그램이 도는 PC에서 띄워지게 되어 있었거든요. 그러니까 stdio 서버는 그걸 띄우는 쪽의 실행 환경에서 명령과 패키지를 찾을 수 있어야 합니다.

이렇게 설정 한 줄이 연결의 길과 서버가 도는 자리를 정합니다. 그 길로 연결한 호스트는 목록을 받아서 도구를 알고, 입력 스키마에 맞춘 값을 넘기고, 정해진 모양의 응답을 받아요.

[SLIDE 40] 스킬과 MCP: 수행 방법과 실행 도구

연결의 길까지 따라왔으니, 다섯 번째 단원에서는 지금까지 따로따로 본 스킬과 MCP를, 한 실행 안에 함께 놓아 보겠습니다. 스킬이 있는데 왜 서버를 따로 연결해야 할까요? 거꾸로, 서버가 있는데 왜 스킬이 필요할까요? 한 실행 안에서 둘의 몫을 나눠 보면 답이 보입니다.

[SLIDE 41] 스킬과 MCP의 역할 구분

둘을 나란히 놓았습니다. 왼쪽 스킬은 수행 방법이고, 오른쪽 MCP는 실행 도구를 잇는 연결이에요.

왼쪽 스킬은 수행 방법을 적은 지침입니다. 사람이 읽고 고칠 수 있는 글이고, 지침 문장 자체가 호출을 실행하지는 않아요. 스킬 묶음에 실행 스크립트가 있어도 그것을 읽고 호출할 실행 주체와 권한이 따로 필요합니다. 원인 진단 도구로 원인 1순위를 얻으라고 적어 둘 뿐이죠.

오른쪽 MCP는 도구를 내주고 호출을 받는 연결입니다. 실제로 일을 하는 건 서버라는 프로그램이에요. 그런데 어떤 순서로 부를지는 정하지 않습니다. 판단 엔진 서버는 원인 진단을 부르면 실행해서 응답할 뿐이에요.

한 문장으로 하면 이렇습니다. 스킬은 무엇을 어떤 순서로 할지를 적고, MCP는 그 글에 적힌 도구를 실제로 부를 수 있게 이어 준다. 이렇게 방법과 실행이 나뉘어 있으니까, 방법을 바꿀 때는 지침을, 접속 위치를 바꿀 때는 연결 설정을 먼저 살펴볼 수 있습니다. 도구의 이름이나 입력 형식까지 바뀌면 지침과 연결을 함께 맞춰야 하고요.

[SLIDE 42] 둘 가운데 하나만 있을 때

이 냉각 대응 업무에서 지침과 도구 중 하나가 빠지면 어떻게 될까요? 첫 카드는 필요한 도구를 연결하지 않은 경우입니다. 지침에 원인을 진단하라고 적혀 있어도 현재 설비 값을 가져올 길이 없으면 이 업무를 끝낼 수 없습니다. 반대로 글만으로 처리할 수 있는 일까지 언제나 MCP가 필요한 것은 아니에요.

둘째 카드는 도구는 있지만 업무 방법을 주지 않은 경우입니다. 호출은 가능해도 어떤 순서로 확인하고 무엇을 제출할지 모델이 그때그때 정할 부분이 커집니다. 방법은 스킬 외에도 프롬프트나 고정된 프로그램으로 줄 수 있어요. 여기서는 같은 방법을 재사용하기 위해 스킬로 관리하는 겁니다.

셋째 카드는 지침과 필요한 도구를 함께 갖춘 경우예요. 어떤 일을 할지와 실제 정보를 얻을 길이 마련됩니다. 그 상태에서도 권한, 입력, 응답을 확인해야 하고, 지침대로 실행됐는지는 기록으로 검토합니다. 둘을 연결한 것만으로 정답이 보장되지는 않아요.

예를 들어 지침에서 생산 오더 조회 도구를 지정했는데 그 도구가 있는 업무 DB 서버를 연결하지 않았다면, 그 호출 경로부터 보완해야 합니다. 반대로 연결과 입력이 정상인데 엉뚱한 업무 순서를 따랐다면 지침을 살펴보겠죠. 문제가 생긴 자리를 가려 고칠 수 있다는 것이 역할을 나눈 실용적인 가치입니다.

[SLIDE 43] 한 실행에서 본 스킬의 단계와 서버의 응답

둘째 쿨러 경보의 실행 기록을 스킬 단계와 대응시켜 놓았습니다. 이 표의 줄은 절차 번호 순서이며 실제 호출의 시간순은 아닙니다. 실제로는 원인 진단 뒤 후보 비교를 먼저 부르고, 그다음 업무 값을 확인했습니다. 표의 왼쪽 칸이 스킬, 가운데와 오른쪽 칸이 서버의 몫이에요.

첫 줄, 2단계 원인 진단은 판단 엔진 서버가 실행해서, 원인 1순위로 쿨러 핀 오염을 돌려줬습니다. 보류 표시가 없었으니, 보류하지 않고 다음 도구로 넘어갔죠.

둘째 줄, 3단계 사실 확인은 업무 DB 서버가 맡았어요. 납기 오더와 출하 대기 로트를 돌려줬죠.

셋째 줄, 4단계 후보 비교는 다시 판단 엔진 서버의 몫이었는데요, 응답이 너무 커서 본문이 에이전트에게 전달되지 않았습니다.

그럼 후보 비교의 결과를 못 받고 어떻게 판단을 냈을까요? 넷째 줄을 보세요. 이 실행은 절차에 없던 조치 효과 예측을 세 번 따로 불러서, 조치 안마다 유온이 어떻게 될지를 받았어요. 그리고 마지막 줄, 5단계의 판단 제출 도구는 서버 안에서 후보 비교를 다시 해서 순위를 매긴 묶음을 제출하고, 판단 번호와 함께 추천과 설명을 돌려줬습니다. 에이전트는 이 설명과 예측 값을 근거로 지는 안의 까닭을 적었고요.

이 표를 위에서 아래로 읽으면 스킬의 방법을 따라가는 거고, 왼쪽에서 오른쪽으로 읽으면 그 방법이 어느 서버의 실행으로 바뀌었는지가 보입니다. 절차 밖의 넷째 줄도 마찬가지예요. 무엇을 더 부를지는 에이전트가 정했지만, 값을 돌려준 건 역시 서버였습니다.

다음 시간에는 이렇게 연결한 도구를 에이전트에게 어디까지 허락할지, 그리고 4단계처럼 응답이 다 오지 못했을 때 무엇을 남겨야 하는지, 연결의 안전 기준을 다룹니다.

[이론 03] 여러 도구를 연결할 때 지켜야 할 안전 기준 이해하기

[SLIDE 44] 도구 권한과 연결 검사

이번 시간에는 여러 도구를 연결할 때 지킬 안전 기준을 다룹니다. 앞 시간에 한 실행 안에서 스킬과 MCP가 각자 무엇을 맡는지 나눠 봤죠. 이번에는 연결한 도구의 권한부터, 파일과 비밀값과 불완전한 응답을 어떻게 다루는지까지 봅니다. 이어서 네 실습에서 무엇을 확인하고 남길지 함께 안내합니다. 실제 실습을 마친 뒤에는 복습과 퀴즈로 돌아옵니다.

여섯 번째 단원부터 시작할게요. MCP로 서버를 하나 더 연결하면, 에이전트가 할 수 있는 일이 늘어나죠. 그런데 잘못했을 때 생기는 일도 같이 늘어납니다. 연결한 도구 가운데 어디까지 에이전트에게 허락해야 할까요? 읽는 도구와 쓰는 도구의 차이부터 보고, 에이전트가 부를 수 있는 도구를 정하는 장치까지 보겠습니다.

[SLIDE 45] 읽는 도구와 쓰는 도구: 잘못 불렀을 때의 차이

도구를 연결할 때는 어떤 데이터를 읽는지와 무엇을 바꾸는지를 먼저 봅니다. 화면 왼쪽은 읽는 도구, 오른쪽은 쓰는 도구예요.

생산 오더 조회는 업무 기록을 바꾸지 않고 값을 가져옵니다. 그렇다고 잘못 불러도 아무 영향이 없는 것은 아니에요. 읽을 필요가 없는 다른 부서의 계약이나 개인정보를 가져오면 정보가 노출될 수 있습니다. 또 잘못된 설비의 값을 받아 판단에 쓰면 이후의 추천도 틀어집니다. 읽기에서도 대상과 범위를 제한해야 하는 이유입니다.

메일 발송처럼 쓰는 도구는 시스템을 바꾸거나 외부에 결과를 남깁니다. 이미 보낸 메일은 요청을 취소한다고 상대의 받은편지함에서 사라지지 않죠. 어떤 변경인지, 누가 허락하는지, 중복 호출을 어떻게 다룰지까지 확인해야 합니다.

이 구분은 도구가 미치는 영향을 살펴보는 출발점입니다. 읽기인지 쓰기인지 표시를 확인하고, 실제 구현과 접근 계정이 그 범위에 맞는지도 봐야 해요. 다음 장에서는 서버가 이 성격을 어떤 표시로 알려 주는지 보겠습니다.

[SLIDE 46] readOnlyHint: 서버가 도구에 다는 읽기 전용 표시

그럼 어떤 도구가 읽는 도구인지는 어떻게 알까요? MCP에는 도구 정의에 붙이는 표시가 있습니다. 화면 첫 줄의 readOnlyHint, 읽기 전용 표시예요.

이 표시는 도구 정의의 주석 칸에 붙고, true면 시스템을 바꾸지 않는 도구라는 뜻입니다. 앞에서 본 부품 견적 도구에도 붙어 있었죠.

둘째 줄을 보세요. 판단 엔진 서버는 판단을 제출하는 두 도구만 빼고 모두 true예요. 그 두 도구가 false인데, 하나는 조치 추천안을, 하나는 구매 추천안을 제출하죠. 셋째 줄, 업무 DB 서버의 도구는 모두 true고요.

그런데 넷째 줄이 중요합니다. 이 표시는 이름 그대로 힌트예요. 서버가 스스로 적은 값이라서, 서버를 만든 사람이 잘못 적을 수도 있고 일부러 속일 수도 있습니다. 그래서 HYD 포털은 표시와 함께 도구 이름까지 한 번 더 검사합니다. 표시는 true인데 이름이 쓰기나 삭제를 뜻하면, 읽는 도구로 자동 인정하지 않습니다. 다만 이름 검사도 잘못된 구현이나 거짓 표시를 모두 밝혀 내는 증명은 아니에요. 판단 엔진 서버의 판단 제출 도구는 아예 표시부터 false죠. 그래서 포털의 시험 화면에서는 부를 수 없고, 처리 건 안의 에이전트만 부릅니다.

읽기 도구 쪽도 표시에만 기대지 않습니다. 넷째 줄 끝에 적은 것처럼, 업무 DB와 시계열 DB를 읽는 서버는 뒤에서 읽기 전용으로 접속해요. 그러니까 표시는 도구를 고르는 기준이고, 실제로 읽기만 하게 막는 일은 서버 뒤의 장치가 함께 맡는 거예요.

[SLIDE 47] 등록한 서버의 도구가 지나는 세 겹

이 표시가 실제로 어디서 쓰이는지는 실습에서 직접 겪게 됩니다. HYD 포털에 서버를 직접 등록하면, 그 서버의 도구는 세 겹을 지나야 에이전트에 닿아요. 그림 위쪽에 번호가 붙은 세 띠가 그 세 겹입니다.

첫째 겹은 서버가 도구에 다는 표시예요. 왼쪽 띠에 도구가 셋 있죠. 읽기로 표시한 도구, 쓰기로 표시한 도구, 아무 표시가 없는 도구입니다.

둘째 겹은 연결 검사예요. 포털이 서버에 실제로 연결해서 도구 목록을 받아 보고, 읽기 표시가 있으면서 이름도 쓰기를 뜻하지 않는 도구만 통과시킵니다. 쓰기로 표시한 도구는 여기서 걸러져요. 표시가 아예 없는 도구도 걸러집니다. 읽는 도구인지 알 수 없으니까요. 다만 HYD에는 표시만 빠진 도구를 강사가 읽기로 확인하는 별도 절차가 있습니다. 쓰기 표시가 있거나 이름이 쓰기를 뜻하는 도구에는 이 예외를 적용하지 않습니다. 그림은 별도 확인이 없는 기본 판정입니다.

셋째 겹이 허용 목록입니다. 검사를 통과한 도구의 이름만 이 목록에 오르고, 오른쪽의 에이전트는 목록에 있는 도구만 불러요. 그림에서 세 도구를 왼쪽부터 따라가 보면, 끝까지 이어지는 건 맨 위의 부품 견적 도구 하나뿐이죠. 나머지 둘은 가운데 띠에서 멈춥니다.

그리고 연결 검사는 한 번 하고 끝나는 게 아닙니다. 서버 설정을 고치면 검사 통과 기록이 사라지고, 다시 검사해야 에이전트에 붙어요. 검사한 설정과 지금 설정이 같을 때만 통과를 인정하는 거죠. 이 검사가 서버 내부의 모든 동작을 시험하거나 이후 변경까지 보증하는 것은 아닙니다. 목록 확인, 호출 시험, 실제 업무 결과 검증의 범위를 나눠 봐야 합니다.

[SLIDE 48] 연결 검사의 실패 사유와 분류

그럼 둘째 겹인 연결 검사를 통과하지 못하면 어떻게 될까요? 실습에서 서버를 등록하다 보면 연결 검사가 실패할 수 있는데, 그럴 때 무엇을 보면 되는지 표로 정리했습니다.

먼저 원칙이 하나 있어요. 연결 검사는 도구 목록을 받지 못했을 때 빈 목록을 성공처럼 돌려주지 않습니다. 실패로 알리고, 무엇이 막혔는지를 분류와 사람이 읽는 사유로 함께 줘요. 표의 왼쪽 칸이 그 분류입니다.

위의 두 줄은 서버를 찾는 단계의 실패예요. 첫째는 접속을 거절당한 경우입니다. 서버가 켜져 있는지, 포트가 맞는지 봐요. 둘째는 이름을 해석하지 못하거나 네트워크 경로에 닿지 못한 경우고요. 서비스 이름이나 호스트 이름을 확인합니다.

셋째 줄이 시간 초과예요. 정해진 시간 안에 응답이 없으면 실패입니다. 포털은 목록을 받을 때도, 도구를 부를 때도 정해진 몇 초까지만 기다리고, 부를 때를 조금 더 길게 잡아 뒀어요. 그 안에 답이 없다면 서버가 느리거나, 방화벽이 막고 있을 수 있습니다.

넷째는 인증 실패예요. 자격 증명이 없거나 만료됐는지, 그 자격에 필요한 접근 권한이 있는지 살핍니다. 사유를 확인하고 자격 증명과 권한을 바로잡은 뒤 다시 검사해요. 다섯째는 주소의 경로가 MCP 끝점이 아닌 경우고요, 여섯째는 stdio 서버의 명령을 찾지 못한 경우예요. 여섯째는 앞에서 본 것처럼, stdio 서버는 그걸 띄우는 쪽 PC에서 실행된다는 점을 떠올리면 됩니다.

이렇게 분류가 있으면 고칠 곳이 좁혀져요. 그냥「연결 실패」라고만 나오면 서버, 주소, 토큰, 경로를 전부 의심해야 하잖아요. 사유가 다음에 확인할 곳을 가리켜 주니까, 실습 중에 막혀도 어디부터 볼지 알 수 있습니다.

[SLIDE 49] 허용 목록(Allowlist): 에이전트가 부를 수 있는 도구의 이름

셋째 겹인 허용 목록이 하는 일은 멈춤 조건에서 이미 봤죠. 사전 허용한 범위를 넘는 도구 요청은 호스트의 권한 검사에서 다룬다고 했습니다. 구체적으로 추가 확인을 요구하는지, 명시적으로 막는지는 실행 정책에 따라 달라요. 화면 왼쪽이 그 목록의 실제 내용이에요.

맨 위 두 줄은 지식 그래프 서버의 조회 도구 둘입니다. 이 서버는 도구 이름을 하나씩 적어서 조회만 허용했어요. 사실 이 서버에는 그래프를 고치는 도구도 있습니다. 그런데 서버 설정에서 읽기 전용으로 띄워서, 그 도구까지 막아 뒀어요.

그 아래 네 줄 끝에 별표가 붙어 있죠? 별표는 그 서버가 내주는 도구 전부라는 뜻입니다. 판단 엔진 서버와 업무 DB 서버들처럼, 미리 검사해 둔 기준 서버에만 별표를 써요. 그래서 판단 엔진 서버의 별표 안에는, 앞에서 본 판단 제출 도구 둘도 들어 있습니다.

맨 아래 줄은 파일 읽기, 스킬 읽기 같은 호스트 자신의 도구입니다.

오른쪽에서 강조한 게 학생이 등록한 서버예요. 학생이 새로 등록한 서버에는 별표를 쓰지 않습니다. 연결 검사에서 읽기로 인정된 도구의 이름만 하나씩 오르고, 읽기로 인정되지 않은 도구는 실행을 막습니다. 모델이 그런 호출을 요청하는 것과 실제로 실행 권한이 주어지는 것은 별개예요. 결국 이 목록이 곧 에이전트가 쓰는 도구의 범위예요.

[SLIDE 50] 쓰기의 세 층: 판단의 제출, 설비 명령, 업무 기록과 발송

별표 안에 쓰기 도구가 들어 있다면, 에이전트는 어디까지 쓸 수 있는 걸까요? 쓰기에도 층이 있습니다. 화면의 카드 세 장이 그 세 층이에요.

첫 카드는 판단의 제출입니다. 판단 엔진 서버의 쓰기 도구 둘이 여기 해당해요. 이 도구들이 쓰는 건 사람이 고를 추천안입니다. 설비 명령이나 발주·작업지시를 실행하는 쓰기와는 구별됩니다. 이 HYD 사례에서 에이전트에 연결된 MCP 쓰기 도구는 판단 제출용 두 개이고, 작업 폴더의 결과 파일도 따로 작성합니다.

두 번째 카드는 설비 명령이에요. 팬 속도를 바꾸는 명령은 담당자가 승인한 뒤에야 만들어집니다.

세 번째 카드는 업무 기록과 메일입니다. 작업지시를 등록하고 메일을 보내는 일이죠. 이것도 승인 뒤의 시스템이 해요. 메일 발송 도구는 아예 에이전트에 연결하지 않습니다.

추천은 에이전트가, 승인은 사람이, 실행은 시스템이 한다는 경계가 있었죠. 이 경계가 지침의 문장에만 있는 게 아닙니다. 어느 도구를 누구에게 연결했는지에도 놓여 있어요. 그래서 스킬에 실수로 메일을 보내라고 적혀 있더라도, 이 MCP 경로에는 메일을 실행할 도구가 없습니다. 같은 효과를 내는 다른 도구나 직접 접속 경로까지 없는지는 전체 실행 권한에서 확인해야 합니다.

[SLIDE 51] 최소 권한 원칙: Principle of Least Privilege

읽기에서도 대상과 범위를 제한해야 한다고 했죠. 이를 일관되게 적용하는 기준이 무엇일까요? 보안에는 이 물음에 답하는 원칙이 있습니다. 최소 권한 원칙이에요.

첫 줄의 정의를 보세요. 최소 권한 원칙은 맡은 일에 필요한 권한만 주고, 그 밖의 권한은 주지 않는다는 원칙입니다. 읽기 도구라도 그 일에 안 쓰이면 연결하지 않아요.

HYD의 업무 DB 서버가 셋으로 나뉜 까닭이 여기 있습니다. 둘째 줄, 정기 정비 계획 에이전트에는 정비 쪽 서버만 연결해서 정비 도구 네 개만 받게 했어요. 셋째 줄, 예비품 구매 에이전트에는 구매 쪽 서버만 연결해서 구매 도구 세 개만 받게 했고요.

두 서버의 도구는 서로 겹치지 않습니다. 그리고 나눈 서버에는 SQL을 자유롭게 쓰는 도구를 두지 않았어요. 그런 도구가 있으면 정해 둔 범위를 넘어 다른 표까지 읽을 수 있거든요. 읽기라도 범위를 좁혀 두면, 에이전트가 잘못 판단했을 때 닿을 수 있는 곳도 좁아집니다.

[SLIDE 52] 파일 범위 · 비밀값 · 불완전한 응답

이제 마지막 이론 단원입니다. 허용 목록과 연결 검사로 에이전트가 부를 도구는 정했어요. 그런데 도구 권한 밖에서 새는 곳은 없을까요? 그리고 도구가 답을 다 주지 못했을 때는 무엇을 남겨야 할까요? 에이전트가 읽고 쓰는 파일의 범위, 서버에 연결할 때 쓰는 비밀값, 불완전한 응답의 처리를 차례로 짚어 보겠습니다.

[SLIDE 53] 파일 접근 범위: 작업 폴더 하나

에이전트는 파일을 읽고 쓰는 도구도 씁니다. 스킬 파일을 읽고, 결과 파일을 쓰죠. 그럼 그 범위는 어디까지일까요?

첫 카드가 범위예요. 에이전트는 실행마다 작업 폴더 하나를 받고, 그 폴더 밖의 파일은 읽거나 고치지 말라는 지시를 받습니다. 카드에 따옴표로 적힌 문장이 그 지시의 원문이에요.

두 번째 카드는 금지입니다. 홈 디렉터리를 찾아보지 말라는 지시가 따로 있어요. 홈에는 프로그램의 설정과 로그인 정보가 들어 있거든요.

세 번째 카드는 결과의 자리예요. 결과를 쓰는 파일은 정해진 경로 하나뿐입니다.

여기서 정확히 말해 둘 게 있어요. 작업 폴더를 나누고 지시하는 것 자체는 운영체제의 강제 격리가 아닙니다. 실제 접근을 막으려면 실행 계정 권한, 허용 경로 검사, 격리 환경 같은 별도의 통제가 필요해요. 지금 화면은 HYD의 지침과 결과 저장 위치를 설명한 것입니다. 그래서 범위를 정하는 것과 함께, 에이전트가 닿을 수 있는 곳에 아예 비밀을 두지 않는 일이 필요해요.

[SLIDE 54] 인증 비밀값: 설정에는 자리표시자, 값은 실행 직전에

그런 비밀 가운데 하나가, 서버에 연결할 때 쓰는 인증 비밀값입니다. 외부 서비스에 연결하려면 토큰 같은 값이 필요하죠. 그런데 이 값을 서버 설정에 그대로 적으면, 설정을 보는 사람 누구에게나 드러나요.

그래서 설정에는 값 대신 자리표시자를 둡니다. 왼쪽 위 설정에서 인증 머리말을 보세요. 토큰이 들어갈 자리에 값이 아니라, 비밀값이라는 표시와 그 값의 이름만 적혀 있죠. 실제 값은 설정과 따로 보관했다가, 실행 직전에만 채웁니다.

값이 준비되지 않았으면 어떻게 할까요? 빈 토큰으로 연결을 시도하지 않아요. 그 서버를 그 실행에서 뺍니다. 그리고 화면이나 기록에 접속 정보가 보일 때는, 왼쪽 아래처럼 값을 별표로 가려요. 연결 방식을 볼 때 지식 그래프 서버의 설정에서 뺐던 비밀번호 칸이, 포털 화면에는 이렇게 가려진 채로 보입니다.

자리표시자를 쓰면 토큰을 바꿀 때도 편합니다. 토큰이 만료돼서 새 값으로 바꿔도, 설정에 적힌 건 자리표시자 그대로잖아요. HYD는 자리표시자가 든 설정으로 검사 지문을 계산하므로, 토큰 값만 바꿔도 기존 검사 표시는 유지될 수 있습니다. 하지만 새 토큰의 계정과 권한까지 예전과 같다는 뜻은 아니에요. 실제 접속과 필요한 권한은 확인해야 합니다.

에이전트 쪽에도 지킬 비밀이 있어요. 오른쪽 넷째 줄입니다. 에이전트 프로그램을 띄우는 쪽은 데이터베이스 접속 문자열과 비밀번호를 갖고 있어요. 그중 워커만 쓸 DB 비밀번호 같은 값은 자식 프로그램에 상속되지 않도록 환경변수에서 지웁니다. 모델 서비스 로그인처럼 실행에 필요한 값은 남는 예외가 있어요. 에이전트가 그 값을 읽으면, 허용 목록을 거치지 않고 데이터베이스에 바로 접속할 수 있거든요. 또 MCP 연결에 필요한 비밀값은 실행 중 설정에 들어갈 수 있고, 종료 뒤 정리하는 코드가 있습니다. 자리표시자와 화면 가림은 노출을 줄이는 방법이지, 실행 중 모든 비밀이

에이전트에게 보이지 않는다는 보장은 아닙니다. 무엇이 어느 과정에 전달되는지까지 확인해야합니다.

[SLIDE 55] 외부 응답은 불완전할 수 있다: 잘린 결과의 실제 사례

도구의 응답을 온전히 읽지 못한 경우에는 무엇을 남겨야 할까요? 표에는 두 쿨러 경보의 기록을 나란히 정리했습니다.

첫 줄이 공통 상황입니다. 두 실행 모두 조치 후보 비교 응답이 12만 자를 넘었고, 실행 환경이 본문을 다른 파일로 보관했습니다. 에이전트가 원래 호출 응답에서 상세 내용을 직접 받지 못한 거예요. 이것은 이 실행 환경의 전달 제한이지, MCP가 모든 응답에 같은 글자 수 한도를 정한다는 뜻은 아닙니다.

둘째 줄의 첫 실행에서는 필요한 항목만 좁혀 다시 뽑겠다는 말이 남았습니다. 이것은 앞으로 할 일에 대한 계획입니다. 그 말만 보고 실제로 다시 조회해서 성공했다고 기록하면 안 됩니다. 실행한 호출과 돌아온 결과가 있어야 그 다음을 말할 수 있어요.

셋째 줄의 다른 실행은 열지 못한 상세 출력이 있음을 마지막 보고에 밝혔습니다. 판단 제출 결과의 설명과 별도로 조회한 예측 값에 근거해 보고했다는 범위도 함께 남겼고요.

받은 결과, 다시 확인하겠다는 계획, 끝내 읽지 못한 범위를 나누는 이유는 보고를 읽는 사람이 무엇을 근거로 판단했는지 알게 하려는 것입니다. 설명이 그럴듯한지만 보지 말고, 그 설명을 뒷받침한 자료가 실제로 전달됐는지를 확인해야 합니다.

[SLIDE 56] 값 없음 · 실패 · 시간 초과 · 잘림의 구분

응답이 기대와 다를 때 결과에 어떻게 적어야 하는지, 표에 네 경우로 정리했습니다.

위의 두 줄은 멈춤 조건에서 이미 구분한 거예요. 정상 조회에서 0행이면 그 조건에 맞는 행을 찾지 못했다고 적습니다. 재고 수량이 0이라는 뜻은 아니에요. 호출 실패나 도구 오류가 확인됐다면 사유를 적고, 0이나 빈 값으로 바꾸지 않습니다.

도구를 연결해서 쓰면 경우가 둘 더 생깁니다. 강조된 셋째 줄이 정해진 시간 안에 응답하지 못한 경우예요. 연결 검사에서 본 시간 초과가, 도구를 부를 때도 생기는 거죠. 업무 DB 조회처럼 시간 제한이 걸린 호출은, 시간을 넘기면 실패로 돌아옵니다. 이때 확인되는 것은 제한 시간 안에 결과를 받지 못했다는 사실입니다. 서버가 작업을 전혀 하지 않았다고 단정할 수는 없어요. 특히 변경을 남기는 도구라면 재시도 전에 처리 상태와 중복 방지 여부부터 확인해야 합니다.

넷째 줄은 응답이 잘린 경우입니다. 본문의 일부만 오거나 아예 전달되지 않으면, 읽지 못한 범위를 밝혀요. 포털의 도구 시험 화면도 같은 원칙을 따릅니다. 결과가 정해진 길이를 넘으면 잘라서 보여 주되, 잘렸다는 표시와 원래 크기를 같이 알려 줘요.

네 경우 모두 지키는 원칙은 하나예요. 모르는 걸 아는 것처럼 적지 않는다. 시간을 넘긴 조회를 0으로 적으면 어떻게 될까요? 모르는 재고가 없는 재고로 바뀌고, 잘못된 발주로 이어질 수 있습니다.

[SLIDE 57] 근거 표시: 호출 기록과 값의 출처

마지막 기준은 근거 표시입니다. 첫째 쿨러 경보의 실행 기록 한 줄을, 이번에는 근거 표시라는 눈으로 다시 읽어 볼게요. 화면 왼쪽이 그 한 줄이에요.

기록에는 언제 불렀고 얼마나 걸렸는지, 어느 서버의 어느 도구를 불렀는지가 남아 있습니다. 판단 엔진 서버의 원인 진단 도구를 불렀고요, 넘긴 입력과 돌아온 원문이 나란히 있어요.

그런데 여기에 겹이 하나 더 있습니다. 오른쪽 맨 아래, 값의 출처예요. HYD의 판단 결과에는 값을 읽은 원천과 조회 근거가 함께 남습니다. 관측 시각과 조회 시각은 같은 뜻이 아니므로 각 항목이 나타내는 시각을 확인해야 해요. 앞서 본 생산 오더의 납기 값은 MES 자료에서 왔다는 것을 따로 추적할 수 있습니다. 왼쪽 원인 진단 한 줄만으로 그 납기 값까지 확인했다고 읽지는 마세요. 판단 엔진의 출처 정보에는 원천과 표, 조건, 실행한 조회문 등이 담깁니다. 모든 외부 MCP 서버가 이런 출처 항목을 자동으로 준다는 뜻은 아닙니다.

서버를 여러 개 연결할수록 이 두 겹의 표시가 중요해집니다. 어느 서버가 준 값인지, 그 서버는 어디서 그 값을 읽었는지가 남아 있어야, 사람이 추천을 승인하기 전에 근거를 되짚을 수 있거든요. 그러니까 도구를 안전하게 연결한다는 건 두 가지예요. 권한 밖으로 새는 곳을 막는 일, 그리고 받은 것과 받지 못한 것을 추적할 수 있게 남기는 일입니다.

[SLIDE 58~59] 실습: HYD 업무 도구의 연결과 호출

지금까지 스킬이 부르는 도구가 MCP로 어떻게 이어지는지, 그리고 그 연결에 어떤 안전 기준이 붙는지를 따라왔죠. 이제 그걸 직접 해 볼 차례입니다. 첫 번째 실습에서는 HYD 포털에서 업무 도구를 직접 연결하고 불러 봅니다. 먼저 이미 등록된 업무 DB 서버와 판단 엔진 서버를 열어서, 연결 방식과 도구 목록을 읽어요. 그다음 실습용으로 제공하는 읽기 전용 서버를 등록하고, 연결 검사를 거쳐서 읽기 도구 하나를 호출합니다. 호출한 도구는 기존 에이전트에 연결하고요.

실습이 끝나면 두 가지가 남습니다. 연결 검사를 통과한 기록, 그리고 도구 이름과 입력 값과 반환된 행이 적힌 호출 기록 한 줄이에요. 쓰기 도구가 왜 호출 대상에서 빠졌는지 앞에서 본 세 겹으로 설명할 수 있으면 성공입니다.

이것이 첫 실습에서 확인할 내용입니다. 이어서 나머지 실습도 살펴본 뒤 실제 화면으로 넘어가겠습니다.

[SLIDE 60~61] 실습: 서로 다른 MCP 서버의 연결과 조합

두 번째 실습은 여러 서버를 조합하는 활동입니다. 첫 실습에서 확인할 포털의 연결 검사와 비교해 보면 차이가 드러납니다. 두 번째 실습에서는 자리를 내 PC의 개발 환경으로 옮겨서, 서로 다른 MCP 서버 여러 개를 연결하고 조합해 봅니다. 이번 실습은 바이브 코딩으로 진행해요. 미리 준비된 파일시스템, Git, 시간 서버를 각각 연결하고, 서버마다 실행 방법과 접근 권한을 정합니다.

연결한 다음에는 서버마다 도구 목록과 호출 결과가 어떻게 다른지 견줘 보겠습니다. 그리고 파일을 읽고, Git의 변경 기록을 확인하고, 시간 정보를 확인하는 일을 한 요청으로 이어서 불러요. 그 조합 결과가 이 실습에서 남는 산출물입니다.

성공 기준은 연결 방식의 차이를 설명하는 거예요. HYD 포털에는 방금 배운 등록·검사 절차가 있습니다. 로컬 코딩 도구에서는 해당 도구가 제공하는 서버 설정과 실행 권한을 확인해야 합니다. 이 차이를 설명할 수 있으면 됩니다.

서버별 결과가 한 요청 안에서 어떻게 사용됐는지도 호출 기록으로 남기겠습니다.

[SLIDE 62~63] 실습: 회의 내용을 후속 업무로 바꾸는 스킬 작성

세 번째 실습은 회의록 스킬 작성입니다. 앞의 두 활동이 도구 연결을 확인하는 실습이라면, 세 번째 실습에서는 이번 회차의 다른 한 축인 스킬로 돌아가서, 회의 내용을 후속 업무로 바꾸는 스킬을 직접 써 봅니다. 예제 회의록을 입력으로 받아서, 결정사항, 담당자, 완료 시점을 뽑아내는 SKILL.md예요. 앞에서 카드 네 장으로 정리한, 스킬을 쓸 때 지킬 네 가지를 그대로 쓰면 됩니다. 언제 쓰는 스킬인지 설명 한 줄로 밝히고, 절차와 결과 항목의 이름을 정하겠습니다.

예외 지침도 꼭 넣습니다. 담당자나 기한이 회의록에 없으면, 지어내지 말고 「미정」으로 표시하게 하는 거예요. 그 스킬로 Markdown과 JSON 작업 목록을 만들면, 그 목록이 산출물로 남습니다. 성공 기준은 넷째 카드에 적었어요. 요약에서 끝나지 않고 누가 언제까지 무엇을 하는지가 항목마다 남고, 항목마다 회의록 원문의 근거 문장이 있으면 성공입니다.

같은 안건이 반복되거나 표현이 다르다고 함부로 합치지 말고, 같은 일인지 판단한 근거와 불확실한 부분을 남깁니다. 작업 목록을 만드는 것과 실제 담당자에게 업무를 발송하는 것도 구분하겠습니다. 이번 활동은 목록을 만들고 원문과 대조하는 데까지입니다.

[SLIDE 64~65] 실습: 대화형 AI 학습 코치 만들기

마지막 실습은 학습 코치입니다. 회의록 활동의 성공 기준으로 항목마다 원문 근거를 붙인다고 했죠. 네 번째 실습에서도 근거를 붙인다는 그 원칙을 그대로 가져가서, 대화형 학습 코치를 만듭니다.

첫 카드가 입력이에요. Notion 학습노트를 Markdown으로 내보낸 파일이나, 제공된 매뉴얼입니다. 둘째 카드가 대화의 차례고요. 입력에서 문항을 만들고, 학생의 답을 받고, 그 답에 정답과 출처를 붙여 되짚어 준 뒤, 다음 문항으로 넘어갑니다. 텍스트 파일을 읽는 방식으로 이 대화를 끝까지 완주하면, 셋째 카드의 산출물인 완주한 대화가 남아요. 피드백마다 정답과 원문의 자리를 붙이고,

실제로 그 원문이 판단을 뒷받침하는지 확인하면 됩니다. 출처 이름만 붙었다고 피드백이 옳은 것은 아니에요. 자료만으로 정답을 정할 수 없다면 그 한계도 밝혀야 합니다. 학습노트는 내보낸 파일로 사용하므로 실서비스 API 연결이 필수는 아닙니다.

네 실습에서 할 일과 남길 결과를 모두 살펴봤습니다. 이제 안내한 순서대로 실제 도구를 연결하고 지침을 작성해 보겠습니다. 실습 뒤에는 결과를 바탕으로 복습과 퀴즈를 이어갑니다.

[SLIDE 66] 복습: 무엇을 어디에 둘 것인가

실습에서 남긴 결과를 바탕으로 이번 회차를 마무리하겠습니다. 복습, 용어 정리, 퀴즈 순서로 갈게요. 네 실습에서 도구를 직접 연결해 불러 보고, 스킬도 직접 써 봤죠. 먼저 복습으로, 이번 회차에 배운 걸 실제 상황에 적용해 보겠습니다. 표의 왼쪽 칸에, 에이전트에게 일을 맡기다 만나는 상황 네 가지를 모았어요. 각 상황에서 먼저 볼 자리를 스킬, SOP, 도구 목록, 자리표시자 가운데서 골라 보세요.

첫째, 정기 정비 경보도 매번 같은 방법으로 처리하게 하고 싶을 때. 둘째, 운전원이 팬을 100 %로 올리는 작업 순서를 찾을 때. 셋째, 판단 엔진 서버의 도구가 셋 늘었을 때. 넷째, 토큰이 필요한 외부 서버를 등록할 때입니다.

가운데 칸과 맞춰 볼게요. 첫째는 스킬입니다. 반복되는 업무의 방법은 이름 있는 지침으로 남기고, 그 일을 맡을 에이전트에 붙이죠. 프롬프트에 매번 적는 게 아니고요.

둘째는 SOP예요. 운전원이 읽는, 설비에 하는 일의 절차죠. 똑같이 「절차」라는 말이 붙어도, 에이전트 스킬에서 찾을 게 아닙니다.

셋째는 도구 목록입니다. 서버가 지금 내주는 기능을 목록에서 확인하고 필요한 권한과 입력 형식도 함께 살펴봅니다. 넷째는 자리표시자예요. 토큰 값은 설정 밖에 두고, 실행 직전에 채웁니다.

돌아보면 네 상황 모두 무엇을 어디에 둘지의 문제였어요. 방법은 스킬에, 사람의 작업 절차는 SOP에, 도구의 정보는 서버가 내주는 목록에, 비밀은 설정 밖에 둡니다. 자리를 맞게 구분하면, 바꿀 일이 생겼을 때 무엇부터 점검할지 알 수 있습니다.

[SLIDE 67] 오늘 다룬 핵심 개념과 주요 용어 정리

상황에 적용해 봤으니, 이번에는 이번 회차에 나온 용어의 뜻을 정리하겠습니다. 표에 아홉 용어가 있고, 영문 정식 명칭은 화면에 같이 적어 뒀어요.

위에서부터 갈게요. 에이전트 스킬은 특정한 일을 수행하는 방법을, 이름을 붙여 적어 둔 에이전트의 지침 묶음입니다. MCP는 AI 애플리케이션이 외부 도구와 데이터에 연결하는 방식을 정한 개방형 프로토콜이고요.

호스트는 모델을 가진 애플리케이션이고, 클라이언트는 그 안에서 서버 하나와 연결을 유지합니다. 서버는 시스템의 기능을 내주는 프로그램, 도구는 서버가 내주는 호출할 수 있는 기능이에요. 입력 스키마는 도구에 넘길 값의 이름과 형을 정한 정의입니다.

stdio와 Streamable HTTP는 연결 방식이죠. 하위 프로세스의 표준 입출력으로 주고받거나, HTTP 요청으로 주고받습니다. 읽기 전용 표시는 도구가 시스템을 바꾸지 않는다고 서버가 스스로 다는 표시예요. 최소 권한 원칙은 맡은 일에 필요한 권한만 주는 원칙이고, 마지막으로 자리표시자는 설정에 비밀값 대신 적어 두는 값의 이름과 자리입니다.

[SLIDE 68] Review Quiz: 핵심 흐름 되짚어보기

용어까지 정리했으니, 배운 내용을 실제 판단에 쓸 수 있는지 퀴즈 세 문제로 확인해 보겠습니다. 문항마다 먼저 이걸 가려 보세요. 문제가 스킬에 있는지, 연결에 있는지, 아니면 응답을 처리하는 데 있는지요. 정답과 까닭은 세 문제를 다 본 뒤에 같이 확인합니다.

[SLIDE 69] Quiz 1. 스킬에 적힌 조회가 실행되지 않은 까닭

첫 번째 문제입니다. 스킬에는 업무 DB의 생산 오더 조회 도구를 직접 호출하라는 단계가 있어요. 그런데 실행해 보니 그 값을 받지 못했습니다. 이 에이전트에 연결된 서버는 지식 그래프 서버와 판단 엔진 서버, 이 둘뿐이에요. 먼저 고칠 곳으로 가장 알맞은 건 무엇일까요?

1번, 스킬의 절차에 납기 오더를 조회하는 도구의 입력 스키마를 옮겨 적는다.
2번, 에이전트에 업무 DB 서버를 연결한다.
3번, 스킬의 설명을 더 자세히 고쳐 쓴다.
4번, 허용 목록에서 판단 엔진 서버의 도구를 뺀다.

[SLIDE 70] Quiz 2. 시간을 넘긴 재고 조회

두 번째 문제입니다. 예비품 재고를 조회하는 호출이, 정해진 시간 안에 끝나지 못했어요. 포털은 시간 초과로 결과를 받지 못했다고 알렸습니다. 결과에 적을 내용으로 가장 알맞은 건 무엇일까요?

1번, 재고가 0개라고 적는다.
2번, 지난 실행에서 본 재고 값을 적는다.
3번, 재고 항목을 결과에서 지운다.
4번, 조회에 실패해 재고를 확인하지 못했다고 적는다.

[SLIDE 71] Quiz 3. 등록한 서버에서 에이전트가 쓸 수 있는 도구

마지막 문제입니다. HYD 포털에 새로 등록한 서버가 도구 세 개를 내줍니다. 설정에 대한 연결 검사는 통과했고, 강사의 별도 읽기 확인은 없는 상황입니다. 행을 읽는 도구는 읽기 전용으로 표시했고, 행을 고치는 도구는 쓰기로 표시했습니다. 내보내기 도구는 표시가 없어요. 연결 검사를 거친 뒤, 에이전트의 허용 목록에 오르는 도구는 무엇일까요?

1번, 행을 읽는 도구 하나.
2번, 행을 읽는 도구와 내보내기 도구.
3번, 세 도구 모두.
4번, 오르는 도구가 없다.

[SLIDE 72] 정답과 판단 기준

세 문제를 다 보셨으면, 고른 답을 근거와 함께 맞춰 보겠습니다. 첫 번째 문제의 정답은 2번, 에이전트에 업무 DB 서버를 연결하는 겁니다. 스킬에는 방법이 이미 적혀 있었어요. 빠진 건 그 방법을 실행할 연결이죠. 납기 오더를 내주는 도구는 업무 DB 서버에 있으니까, 지정된 조회를 수행하려면 그 서버를 연결하고 호출 권한과 입력을 확인해야 합니다. 1번과 3번은 스킬을 고치는 일이라서 빠진 연결을 채우지 못하고요, 4번은 남아 있던 도구까지 줄여 버립니다.

두 번째 문제의 정답은 4번, 조회에 실패해서 재고를 확인하지 못했다고 적는 겁니다. 실패한 조회는 재고가 없다는 걸 알려 주지 않았어요. 재고를 모른다는 것만 알려 줬죠. 1번은 모르는 걸 없는 걸로 바꿨고, 2번은 지난 값을 지금 값처럼 썼고, 3번은 확인하지 못했다는 사실까지 지워 버렸습니다.

세 번째 문제의 정답은 1번, 행을 읽는 도구 하나예요. 이 문제의 HYD 기본 판정에서는 읽기 표시가 있고 이름도 쓰기가 아닌 도구만 읽기로 인정합니다. 고치는 도구는 쓰기로 표시돼 있으니 걸러지고, 내보내기 도구는 표시가 없어서 읽는 도구인지 알 수 없으니 걸러져요. 2번을 골랐다면, 표시가 없는 도구를 읽기로 본 겁니다.

여기까지가 이번 회차입니다.

다음 회차에서는 에이전트가 멈추고 사람에게 물어야 하는 순간을 다룹니다. 사람의 승인과 거절이 업무 결과를 어떻게 바꾸는지 확인하고, 요구사항이 바뀌어도 기존 기능을 지키는 방법도 살펴볼 거예요. 수고 많으셨습니다. 감사합니다.